ACADEMIC TASK GOVERNANCE AND CODE ASSESSMENT PLATFORM

A PROJECT REPORT

Submitted by

THARUNKUMAR K **(922524205171)**

SANTHOSHKUMAR S **(922524205148)**

in partial fulfillment for the award of the degree

of

BACHELOR OF TECHNOLOGY

in

INFORMATION TECHNOLOGY

V.S.B. ENGINEERING COLLEGE, KARUR-639111

(AN AUTONOMOUS INSTITUTION)

ANNA UNIVERSITY : CHENNAI - 600 025

NOV-DEC 2026

ANNA UNIVERSITY : CHENNAI 600 025

BONAFIDE CERTIFICATE

Certified that this project report titled **"ACADEMIC TASK GOVERNANCE AND CODE ASSESSMENT PLATFORM"** is the bonafide work of **K. THARUNKUMAR (922524205171)** and **S. SANTHOSHKUMAR (922524205148)** who carried out the project work under my supervision.

SIGNATURE

Mr. S. NELSON, M.E.
SUPERVISOR
Assistant Professor
Department of Information Technology
V.S.B. Engineering College
Karur – 639 111.

SIGNATURE

Dr. P. AROCKIA MARY, M.E., Ph.D.
HEAD OF THE DEPARTMENT
Department of Information Technology
V.S.B. Engineering College
Karur – 639 111.

Submitted for Anna University Project Viva-Voce held on _____

INTERNAL EXAMINER **EXTERNAL EXAMINER**

ACKNOWLEDGEMENT

First and foremost, we express our profound thanks to our parents for providing us with a supportive environment and continuous encouragement for carrying out this project. We wish to express our sincere gratitude to our founder and Chairman, Shri. V. S. BALSAMY, B.Sc., L.L.B., for his dedication and vision in educating us in this premier institution.

We extend our deep gratitude to **Dr. C. VENNILA, M.E., Ph.D.**, Principal, and **Dr. T. S. KIRUBA SHANKAR, M.E., Ph.D.**, Vice Principal, V.S.B. Engineering College, Karur, for their encouraging support and academic leadership throughout the course of this project.

We express our sincere indebtedness to **Dr. P. AROCKIA MARY, M.E., Ph.D.**, Head of the Department, Department of Information Technology, for her constant support, valuable suggestions, and department facilities.

We are deeply grateful to our Project Supervisor, **Mr. S. NELSON, M.E.**, Assistant Professor, Department of Information Technology, for his continuous guidance, technical insights, and patience throughout the conceptualization and development of our project.

Our sincere thanks to all the faculty members of the Department of Information Technology and our fellow students for their constructive feedback and assistance in the successful completion of this project.

Finally, we bow before Almighty God, who guides our paths and blesses our endeavours with grace and wisdom for the successful completion of this project work.

DEPARTMENT OF INFORMATION TECHNOLOGY

Vision of the Institution:

We endeavour to impart futuristic technical education of the highest quality to the student community and to inculcate discipline in them with self-confidence and thus we prepare them for life as responsible citizens to uphold human values, to be of service at large and establish the Institution as a centre of academic excellence of global standards.

Mission of the Institution:

We transform persons into personalities by state-of-the-art infrastructure, time consciousness, quick response and best practices through continuous assessment and advice.

Vision of the Department:

To provide professional computing training to make competent engineers and thus prepare them to work in the emerging trends of the information technology field.

Mission of the Department:

1. Producing quality IT engineers with good technical knowledge by effective teaching.
2. Making competent engineers adapt to the dynamic needs of the industries.
3. Developing extensive learning skills in academic studies.
4. Inculcating strong ethical values and professionalism to serve society with responsibility.

PROGRAM EDUCATIONAL OBJECTIVES (PEOs)

PEO1: To make graduates proficient in utilizing the fundamental knowledge of various streams in engineering and technology.

PEO2: To enrich graduates with core competencies necessary for applying computing technologies in the context of business enterprise.

PEO3: To enable graduates to think logically and pursue lifelong learning to understand technical issues related to computing systems and provide optimal solutions.

PEO4: To enable graduates to develop software and hardware systems by understanding the importance of social, business, and environmental needs in context.

PROGRAM OUTCOMES (POs)

PO1. **Engineering knowledge:** Apply the knowledge of mathematics, science, engineering fundamentals, and an engineering specialization to the solution of complex engineering problems.

PO2. **Problem analysis:** Identify, formulate, review research literature, and analyze complex engineering problems reaching substantiated conclusions using first principles of mathematics, natural sciences, and engineering sciences.

PO3. **Design/development of solutions:** Design solutions for complex engineering problems and design system components or processes that meet the specified needs with appropriate consideration for public health and safety, cultural, societal, and environmental factors.

PO4. **Conduct investigations of complex problems:** Use research-based knowledge and methods including design of experiments, analysis and interpretation of data, and synthesis of information to provide valid conclusions.

PO5. **Modern tool usage:** Create, select, and apply appropriate techniques, resources, and modern engineering and IT tools including prediction and modeling to complex engineering activities with an understanding of limitations.

PO6. **The engineer and society:** Apply reasoning informed by contextual knowledge to assess societal, health, safety, legal, and cultural issues and the consequent responsibilities relevant to professional engineering practice.

PO7. **Environment and sustainability:** Understand the impact of professional engineering solutions in societal and environmental contexts, and demonstrate knowledge of and need for sustainable development.

PO8. **Ethics:** Apply ethical principles and commit to professional ethics, responsibilities, and norms of engineering practice.

PO9. **Individual and team work:** Function effectively as an individual, and as a member or leader in diverse teams, and in multidisciplinary settings.

PO10. **Communication:** Communicate effectively on complex engineering activities with the engineering community and with society at large, such as comprehending and writing effective reports and design documentation, making effective presentations, and giving and receiving clear instructions.

PO11. **Project management and finance:** Demonstrate knowledge and understanding of engineering and management principles and apply these to one's own work, as a member and leader in a team, to manage projects in multidisciplinary environments.

PO12. **Life-long learning:** Recognize the need for, and have the preparation and ability to engage in independent and life-long learning in the broadest context of technological change.

PROGRAM SPECIFIC OUTCOMES (PSOs)

PSO1: Apply mathematical and computing knowledge to identify and provide solutions for computing problems.

PSO2: Design and develop computer programs and projects using current software technologies.

PSO3: Apply standard engineering practice for software development to deliver quality software products.

ABSTRACT

In technical higher education institutions, departmental academic task governance, continuous laboratory evaluation, and competitive programming velocity encounter operational challenges. Course deliverables are frequently fragmented across shared cloud spreadsheets, messaging channels, and physical observation notebooks. This fragmentation causes grading latency, submission tracking errors, and an absence of unified visibility into daily student problem-solving progress. Additionally, academic institutions require structured methods to assess multi-language coding competency alongside non-invasive evaluation monitoring.

To address these challenges, this project presents **VSBEC IT Vault**, an integrated academic task governance, competitive coding telemetry, and assessment platform engineered for departmental deployment. The system combines a responsive single-page frontend built with **React 19**, **Vite 6**, and **Tailwind CSS 4** with a **Node.js 22 LTS** and **Express 4.21** application service layer (maintaining backward compatibility with the Node.js 20 pilot runtime), connecting to a **PostgreSQL** database hosted on Supabase with transaction-mode connection pooling to handle concurrent requests.

The platform is designed around three primary technical subsystems: **(1) Automated Institutional Data Archival and Backup Engine**: Scheduled GitHub Actions workflows export daily student activity reports, generate structured JSON relational snapshots, and establish disaster recovery pipelines with an 8-hour synchronization cadence; **(2) High-Availability Transactional Email Dispatch**: An account-pooling engine managing multiple Brevo API credentials with automated fallback to secondary SMTP relays, supporting 6-digit one-time password (OTP) verification and credential recovery workflows; and **(3) Sandboxed Multi-Language Compiler Engine**: Local execution runtimes for C, C++, Java, and Python with a 3-second timeout trap (3000 ms), active 64 KB output stream bounds, and automated failover to the Judge0 CE API, supported by client-side Picture-in-Picture (PIP) webcam display and tab-switch telemetry event logging.

During the preliminary 2025–2026 academic pilot evaluation and the 2026–2027 institutional deployment in the Department of Information Technology at V.S.B. Engineering College, the platform supported **369 enrolled students** across 6 departmental sections (Class III IT Sections A, B, and C [186 students], and Class IV IT Sections A, B, and C [183 students]), engaging **365+ active student participants** and achieving an estimated faculty grading workload reduction of 15+ hours weekly. The project was submitted to the Ministry of Education's Smart India Hackathon (SIH 2026) under problem statement SIH26044, receiving institutional finalist nomination. The project outcomes correlate directly with **PO1**, **PO2**, **PO3**, **PO4**, **PO5**, **PO8**, **PO10**, **PO12**, and **PSO1, PSO2, PSO3**.

TABLE OF CONTENTS

CHAPTER NO.	TITLE	PAGE NO.
	ABSTRACT	viii
	LIST OF FIGURES	xii
	LIST OF ABBREVIATIONS	xiii
1.	**INTRODUCTION**	**1**
	1.1 BACKGROUND AND DOMAIN SIGNIFICANCE	2
	1.2 TECHNICAL MOTIVATION AND PROBLEM STATEMENT	2
	1.3 PROJECT OBJECTIVES	3
	1.4 ORGANIZATION OF THE REPORT	3
2.	**LITERATURE SURVEY**	**4**
3.	**SYSTEM ANALYSIS**	**7**
	3.1 EXISTING SYSTEM	8
	3.2 EXISTING SYSTEM DISADVANTAGES	8
	3.3 PROPOSED SYSTEM	9
	3.4 PROPOSED SYSTEM ADVANTAGES	9
4.	**SYSTEM SPECIFICATION**	**11**
	4.1 HARDWARE REQUIREMENTS	12
	4.2 SOFTWARE REQUIREMENTS	13
	4.3 ARCHITECTURAL INTEGRATION	13
5.	**IMPLEMENTATION**	**14**
	5.1 SYSTEM ARCHITECTURE	15
	5.2 MODULES USED	16

CHAPTER NO.	TITLE	PAGE NO.
	5.3 MODULES DESCRIPTION	17
	5.3.1 Role-Based Access Control & Row-Level Security	17
	5.3.2 Role-Scoped Verification Engine & Cloud Asset Lifecycle	18
	5.3.3 Multi-Language Sandboxed Compiler Engine	18
	5.3.4 LeetCode GraphQL & GitHub Telemetry Daemons	19
	5.3.5 Automated Disaster Recovery and Backup Workflow	19
	5.3.6 High-Availability Multi-Account Brevo Email Engine	21
	5.3.7 Tri-Channel Notification Dispatcher	21
	5.3.8 Picture-in-Picture (PIP) Proctoring Telemetry	21
	5.3.9 Corporate Placement Readiness Index	21
6.	**SYSTEM DESIGN**	**22**
	6.1 INTRODUCTION	23
	6.2 CLASS DIAGRAM	23
	6.3 USE CASE DIAGRAM	25
	6.4 SEQUENCE DIAGRAM	26
	6.5 ACTIVITY DIAGRAM	28
7.	**SYSTEM TESTING**	**29**
	7.1 INTRODUCTION AND TEST ENVIRONMENT	30
	7.2 TESTING METHODOLOGIES	30
	7.3 TEST CASES AND EMPIRICAL VALIDATION	31
	7.4 EMPIRICAL RESULTS AND SYSTEM RELIABILITY METRICS	31

CHAPTER NO.	TITLE	PAGE NO.
8.	**FEASIBILITY STUDY**	**32**
	8.1 FEASIBILITY ANALYSIS	33
	8.1.1 Technical Feasibility	33
	8.1.2 Economic Feasibility	33
	8.1.3 Operational Feasibility	34
	8.2 ENGINEERING METHODOLOGY AND AGILE DEVOPS	34
9.	**CONCLUSION AND FUTURE ENHANCEMENT**	**35**
	9.1 CONCLUSION	36
	9.2 PROPOSED FUTURE ENHANCEMENTS	37
	9.3 CONCLUDING REMARKS	37
APPENDIX	**SOURCE CODE & SCREENSHOTS**	**38**
	Listing 1: Sandbox Execution Engine	39
	Listing 2: Judge0 Cloud Failover	40
	Listing 3: LeetCode & GitHub Telemetry Daemons	41
	Listing 4: Role-Scoped Submission Verification Controller	42
	Appendix B: Screenshots of System Interfaces	44
	REFERENCES	**47**
	BIBLIOGRAPHY	**48**

LIST OF FIGURES

FIGURE NO.	TITLE	PAGE NO.
5.1	SYSTEM ARCHITECTURE DIAGRAM	15
6.1	CLASS DIAGRAM	24
6.2	USE CASE DIAGRAM	26
6.3	SEQUENCE DIAGRAM	27
6.4	ACTIVITY DIAGRAM	28
B.1	STUDENT & HOD CODING PROGRESS WITH GITHUB TRACKER AND LEADERBOARD	44
B.2	MONACO CODE COMPILER & COLLABORATIVE LIVE TEACHING HUB	45
B.3	COORDINATOR WORKSPACE & VERIFICATION DASHBOARD	46
B.4	CORPORATE HR PLACEMENT READINESS RATING PORTAL	46

LIST OF ABBREVIATIONS

S.NO.	ABBR.	EXPANSION
1.	API	Application Programming Interface
2.	AST	Abstract Syntax Tree
3.	CGPA	Cumulative Grade Point Average
4.	CI/CD	Continuous Integration and Continuous Deployment
5.	CSV	Comma-Separated Values
6.	DFD	Data Flow Diagram
7.	FSM	Finite State Machine
8.	GraphQL	A Query Language for APIs (API Data Query Specification)
9.	HMAC	Hash-based Message Authentication Code
10.	HOD	Head of the Department
11.	IPC	Inter-Process Communication
12.	IST	Indian Standard Time (UTC+05:30)
13.	JWT	JSON Web Token
14.	OTP	One-Time Password
15.	PIP	Picture-in-Picture
16.	POSIX	Portable Operating System Interface
17.	PRI	Placement Readiness Index
18.	RBAC	Role-Based Access Control
19.	RLS	Row-Level Security
20.	RPO	Recovery Point Objective
21.	RTO	Recovery Time Objective
22.	SHA	Secure Hash Algorithm
23.	UI/UX	User Interface and User Experience
24.	UML	Unified Modeling Language
25.	VAPID	Voluntary Application Server Identification
26.	WebRTC	Web Real-Time Communication

INTRODUCTION

CHAPTER 1

INTRODUCTION

1.1 BACKGROUND AND DOMAIN SIGNIFICANCE

In engineering education, effective governance of continuous academic evaluations and laboratory deliverables is vital for student learning. Computing curricula demand that students actively engage with practical programming assignments, competitive problem solving, and industry technical assessments.

However, departmental administration frequently relies on disconnected communication channels, including generic messaging groups, shared spreadsheets, and physical notebooks. This introduces operational friction: student submissions are misattributed, grading backlogs accumulate, and departments lack an auditable review history.

Concurrently, software engineering recruitment emphasizes demonstrated coding competency on platforms such as LeetCode and version-controlled histories on GitHub. To address these operational and pedagogical gaps, this project develops **VSBEC IT Vault**, an integrated platform for academic task governance, competitive telemetry harvesting, and proctored coding assessments.

1.2 TECHNICAL MOTIVATION AND PROBLEM STATEMENT

While general-purpose Learning Management Systems (LMS) such as Moodle and Google Classroom excel at distributing course syllabi, they exhibit specific limitations when applied to computing education:

1. **Absence of Integrated Code Evaluation:** General systems treat program files as passive attachments, requiring manual local download, environment setup, and execution.
2. **Single-Tier Role Models:** Standard portals lack granular Role-Based Access Control (RBAC) with scoped peer coordinator auditing and faculty advisor verification.
3. **Unsynchronized Data Backups:** Academic records stored in ephemeral databases lack automated offsite snapshot pipelines and structured archival exports.
4. **Isolation from External Telemetry:** Platforms lack automated interfaces with competitive programming APIs, leaving faculty without longitudinal insight into student practice.

1.3 PROJECT OBJECTIVES

To address these technical and operational requirements, the **VSBEC IT Vault** project defines six concrete engineering objectives:

1. **Centralized Academic Task Governance:** Design and implement an auditable role-scoped verification hierarchy (Student Coordinator, Faculty Class Advisor, and Head of Department) with cloud asset management and automated Excel summary generation.
2. **Sandboxed Multi-Language Compiler Engine:** Engineer a sandboxed execution runtime supporting C, C++, Java 17, and Python 3 with a 3-second timeout trap (3000 ms), active 64 KB output buffer enforcement, and automated fallback to the Judge0 CE cloud execution runtime.
3. **Automated Telemetry Harvesting Daemons:** Construct scheduled background services that query LeetCode GraphQL and GitHub REST APIs daily to compute problem solves, active streaks, and departmental leaderboards based on Indian Standard Time.
4. **Automated Disaster Recovery and Relational Snapshot Archival:** Deploy automated scheduled workflows to export daily CSV/JSON student progress reports and generate relational database snapshots to satisfy an operational Recovery Point Objective (RPO) ≤ 8 hours.
5. **High-Availability Transactional Notification Engine:** Implement a multi-account email dispatch engine with automated failover for 6-digit OTP delivery, alongside Telegram bot webhooks and Web Push notifications.
6. **Non-Invasive Browser Evaluation Telemetry:** Integrate Picture-in-Picture (PIP) webcam display, fullscreen mode monitoring, and tab-switch event traps to log student integrity events during assessments for subsequent faculty review.

1.4 ORGANIZATION OF THE REPORT

The remainder of this report is organized as follows: Chapter 2 reviews relevant academic and industry literature. Chapter 3 analyzes existing system limitations and formalizes the proposed architecture and comparison matrix. Chapter 4 specifies hardware and software requirements. Chapter 5 details the core algorithmic, database, and telemetry implementation. Chapter 6 presents formal Unified Modeling Language (UML) system designs. Chapter 7 validates test cases and testing methodologies. Chapter 8 evaluates technical, economic, and operational feasibility. Chapter 9 presents concluding remarks and planned future enhancements. Appendix A provides representative source code listings, Appendix B provides system interface screenshots, followed by References and Bibliography.

LITERATURE SURVEY

CHAPTER 2

LITERATURE SURVEY

"Development and Classroom Evaluation of a Static-Assessment Programming Practice Platform" by Jeon, J., Jang, J., and Jung, S. K. (2026) [1] – The authors investigated a web-based programming practice platform (Code-Assist-Lab) utilizing static code evaluation paradigms, specifically abstract syntax tree (AST) inspection, cyclomatic complexity calculations via Radon, and style linting via Flake8. Conducted as a targeted pilot with 20 gifted middle-school students in a specialized winter computing program at Kyungpook National University (KNU), the study demonstrated that static evaluation reduces server-side execution risks and infrastructure overhead by avoiding arbitrary dynamic execution. However, static analysis alone cannot verify dynamic runtime behavior, handle edge-case execution errors, or enforce process-level execution boundaries. Furthermore, being restricted to a small cohort of gifted pre-college learners, the platform did not investigate or incorporate role-scoped institutional review hierarchies, continuous collegiate laboratory workflows, or longitudinal telemetry tracking across multi-section engineering cohorts.

"Integration of Multiple Sources to Anticipate Student Performance Using Learning Analytics" by Moreno-Marcos, P. M. et al. (2025) [2] – This research explored predictive learning analytics in higher education by consolidating heterogeneous data streams, including Learning Management System (LMS) clickstream telemetry, SPOC video viewing frequencies, and historical grade records. The authors reported that multi-source feature extraction improved early prediction of examination outcomes. However, the study's analytical model operated primarily as a post-hoc reporting system without immediate closed-loop intervention mechanisms, automated notifications, or synchronized practical programming environments to assist struggling students in real time.

"Shaking The Silos: Impact of Sequential Live Coding on Students' Performance and Perceptions" by Cho, K., Ali, S. F., Bang, S. J., and Anwar, S. (2024) [3] – The authors evaluated Sequential Live Coding (SLC), an instructional method where students take turns writing and explaining code in front of peers during class sessions. While their empirical findings showed enhanced classroom engagement and conceptual retention during instructor-led sessions, the methodology was confined to synchronous classroom activities. It did not provide automated longitudinal tracking of independent practice on competitive platforms (such as LeetCode or GitHub) or an asynchronous verification pipeline for continuous laboratory deliverables.

To contextualize the technical challenges and define the functional scope of the proposed system, Table 2.1 summarizes recent literature, noting the primary methodologies investigated by the authors and the operational limitations identified for institutional task governance.

S.NO	PAPER TITLE	AUTHOR(S)	YEAR	IDENTIFIED SCOPE & OPERATIONAL GAPS
1	Development and Classroom Evaluation of a Static-Assessment Programming Practice Platform	Jeon J. et al.	2026	Focused on static analysis in a targeted 20-student gifted pilot at KNU; lacks dynamic runtime verification and role-scoped collegiate review.
2	Integration of Multiple Sources to Anticipate Student Performance Using Learning Analytics	Moreno-Marcos P.M. et al.	2025	Demonstrates predictive modeling from historical LMS data; lacks automated real-time intervention and integrated coding practice environments.
3	Shaking The Silos: Impact of Sequential Live Coding on Students' Performance and Perceptions	Cho K., Ali S.F., Bang S.J., Anwar S.	2024	Examines synchronous classroom live coding; does not track longitudinal external telemetry (LeetCode/GitHub) or asynchronous lab workflows.

Table 2.1: Comparative Summary of Literature Survey and Identified Scope

Synthesis of Literature Gaps and Proposed System Rationale:

Review of the literature reveals that existing educational platforms generally address isolated components of computing education: static code inspection without dynamic evaluation (Jeon et al. [1]), historical predictive modeling without real-time intervention (Moreno-Marcos et al. [2]), or synchronous active learning without automated continuous tracking (Cho et al. [3]).

The proposed **Academic Task Governance and Code Assessment Platform** integrates these dimensions into a unified workflow: (1) a multi-language compiler execution environment supporting C, C++, Java, and Python with automated 3-second timeout limits, output caps, and Judge0 cloud fallback for runtime verification, building upon online judge models reviewed by Wasik et al. [4]; (2) a multi-level Role-Based Access Control (RBAC) governance framework following principles formalized by Sandhu et al. [5]; (3) background daemons that harvest LeetCode GraphQL and GitHub GraphQL telemetry to track ongoing practice velocity; and (4) an auditable role-scoped verification workflow (Student Coordinator, Faculty Class Advisor, and Head of Department) with multi-channel notifications (Telegram, Web Push, and email) to support timely feedback and academic oversight.

SYSTEM ANALYSIS

CHAPTER 3

SYSTEM ANALYSIS

3.1 EXISTING SYSTEM

In prevailing departmental operating models prior to platform adoption, task governance, programming assignment verification, and student career readiness tracking were conducted through disjointed mechanisms. Faculty members and student coordinators relied on a patchwork of consumer tools, including Google Forms, WhatsApp broadcast channels, shared spreadsheets, and printed observation notebooks. While these tools offered initial convenience, their complete lack of architectural integration introduced systemic operational challenges:

1. **Deliverable Fragmentation and Auditing Gaps:** Student submissions were scattered across personal drives and chat messages without a centralized, auditable trail of review timestamps and feedback.
2. **Absence of Native Sandboxed Code Compilation:** Legacy workflows treated source code as passive attachments, requiring evaluators to download, configure dependencies, and compile code locally.
3. **Algorithmic Telemetry Blind Spot:** Students practiced on LeetCode and GitHub independently without faculty having continuous visibility into solve velocity, difficulty distributions, or daily streaks.
4. **Unreliable Alerting and High Deadline Miss Rates:** Deadlines communicated via broadcast channels suffered from low open rates, leading to missed submissions and administrative friction.

3.2 EXISTING SYSTEM DISADVANTAGES

A deep architectural audit reveals several critical vulnerabilities in the legacy operating model:

1. **Zero Automated Disaster Recovery:** Departmental spreadsheets lacked automated offsite backup mechanisms, risking permanent data loss during local hardware failures.
2. **Single-Tier Insecure Authentication:** Legacy portals employed flat access controls without multi-role segregation, exposing grade data to unauthorized manipulation.
3. **Unoptimized Storage Pipelines:** Submissions uploaded directly to application servers as local files exhausted disk capacity and risked directory traversal.
4. **Absence of Assessment Telemetry:** Online assessments conducted via generic forms provided zero detection of tab switching or window defocusing.

3.3 PROPOSED SYSTEM

The proposed **VSBEC IT Vault** platform addresses the core operational limitations of the legacy setup by establishing an integrated, cloud-native architecture engineered specifically for the Department of Information Technology at V.S.B. Engineering College.

The platform is structured across five synchronized engineering pillars: First, a responsive single-page web client built with React 19, Vite 6, and Tailwind CSS provides customized role dashboards for Supreme Admins, HOD, Faculty Class Advisors, Student Coordinators, Students, and Industry Partners. Second, a resilient Node.js 22 LTS and Express 4.21 application layer enforces strict Role-Based Access Control (RBAC) via cryptographically signed JSON Web Tokens (JWT) and Bcrypt hashing. Third, an enterprise PostgreSQL database hosted on Supabase utilizes dynamic transaction-mode connection pooling on port 6543, eliminating serverless connection exhaustion. Fourth, a multi-language sandboxed compiler engine executes student code in C, C++, Java 17, and Python 3 inside process wrappers with 3000ms execution timeout traps, stream output bounds, and failover to the Judge0 CE cloud API. Fifth, an automated disaster recovery workflow running on GitHub Actions executes at an 8-hour cadence (07:50 AM, 03:50 PM, and 11:55 PM IST), exporting complete student progress reports, database JSON snapshots, and maintaining versioned archival backups to achieve an operational Recovery Point Objective (RPO) target ≤ 8 hours.

3.4 PROPOSED SYSTEM ADVANTAGES AND COMPARISON MATRIX

The proposed system delivers measurable operational improvements across departmental governance, academic velocity, and assessment integrity. Table 3.1 details a comprehensive feature-by-feature comparison between traditional academic tools and the verified capabilities of the VSBEC IT Vault platform.

Dimension / Feature	Traditional Departmental Workflow (Forms / Sheets)	Proposed VSBEC IT Vault Architecture
Role-Based Access Control (RBAC)	Flat single-tier or two-tier permissions; no peer coordinator verification layer.	Granular 6-role hierarchy (`SUPREME_ADMIN`, `HOD`, `CLASS_ADVISOR`, `COORDINATOR`, `STUDENT`, `INDUSTRY`).
Code Evaluation Runtime	Passive file upload; manual compiler installation and local execution required.	Multi-language sandbox (C, C++, Java 17, Python 3) with 3000ms timeout traps and Judge0 failover.
External Telemetry Sync	None; external coding practice remains invisible to faculty advisors.	Automated daemons polling LeetCode GraphQL and GitHub REST events every 24 hours.
Disaster Recovery & Backups	Manual file copies; no automated offsite replication or versioned snapshots.	Scheduled GitHub Actions executing at an 8-hour cadence exporting JSON snapshots with an 8-hour RPO.
Transactional Notifications	Unreliable broadcast emails; low open rates and missed deadlines.	Multi-channel notifications via Brevo email pooling (5 nodes), Telegram bot webhooks, and Web Push.
Assessment Integrity & Proctoring	None; open to unauthorized tab switching and external resource access.	Picture-in-Picture webcam monitoring, fullscreen monitoring, and tab-switch blur event traps.
Asset Management	Local server filesystem storage; risk of server disk exhaustion.	Cloudinary cloud storage with automated optimization and secure URL persistence.

Table 3.1 Comparative Analysis: Traditional Systems vs. VSBEC IT Vault

SYSTEM SPECIFICATION

CHAPTER 4

SYSTEM SPECIFICATION

4.1 HARDWARE REQUIREMENTS

The **VSBEC IT Vault** platform is architected with a decoupled multi-tier topology: the client frontend is served via Vercel Edge CDN; the backend API and compiler processes deploy to a dedicated Node.js service host (tested on Render); database storage utilizes Supabase managed PostgreSQL; and media uploads stream directly to Cloudinary. For the departmental pilot phase, standard managed developer tiers were leveraged, whereas Table 4.2 defines the recommended production sizing for full institutional rollout.

Client Side (Student / Faculty Workstation / Mobile Device):

Component	Specification / Threshold
Device Architecture	Laptop / Desktop PC / Android or iOS Smartphone
Processor	Dual-Core 2.0 GHz or higher (x86_64 or ARM64)
System Memory (RAM)	Minimum 4 GB RAM (8 GB recommended for Monaco IDE)
Available Storage	Minimum 250 MB free disk space for browser caching
Peripheral Hardware	Standard Web Camera and Microphone (for Proctored Tests)
Display Resolution	1366×768 minimum (1920×1080 recommended)
Network Interface	Stable 4G/5G mobile connection or Wi-Fi (minimum 1 Mbps)

Table 4.1: Client-Side Hardware Specifications

Server Side (Recommended Production Cloud Infrastructure & Database Host):

Infrastructure Layer	Recommended Production Sizing
Server Compute	Vercel Serverless Edge CDN / Dedicated Node.js 22 LTS Host
Processor (Virtual CPU)	Minimum 2 vCPUs @ 2.4 GHz
Memory (RAM)	2 GB Dedicated RAM (Dynamically Scalable to 8 GB)
Persistent Storage	Cloud-hosted SSD (Supabase PostgreSQL Enterprise Tier)
Network Throughput	Gigabit Ethernet Interface (1 Gbps symmetric)
External Media Storage	Cloudinary Secure Object Storage (Global CDN)

Table 4.2: Recommended Institutional Production Cloud Server Specifications

4.2 SOFTWARE REQUIREMENTS AND EXACT DEPENDENCY MANIFEST

All software libraries and execution environments were audited directly against the project's dependency manifest in `package.json`:

Frontend Client Environment:

Subsystem / Library	Verified Codebase Version / Specification
Web Browser Compatibility	Google Chrome 110+, Firefox 115+, Edge 110+, Safari 16+
Single-Page Framework	React 19.0.0 with React DOM 19.0.0
Build Tool & Bundler	Vite 6.2.0 (ESM mode with Hot Module Replacement)
Language Standard	TypeScript 5.8 (Strict type checking enabled)
UI Styling & Design System	Tailwind CSS 4.1.14 with @tailwindcss/vite
Iconography & Animations	Lucide React 0.546.0, Motion 12.23.24
In-Browser Code Editor	Monaco Editor (@monaco-editor/react 4.7.0)

Table 4.3: Frontend Subsystems and Verified Dependencies

Backend Application & Data Layer:

Service / Component	Verified Codebase Version / Specification
Runtime Engine	Node.js 22.x LTS (Maintenance LTS, backward compatible with 20.x)
Web Application Framework	Express.js 4.21.2 with CORS 2.8.6 and Gzip Compression
Relational Database Driver	PostgreSQL (pg 8.22.0) with custom type parsers
Authentication & Hashing	JSON Web Tokens (jsonwebtoken 9.0.3), Bcrypt.js 3.0.3
Media Asset Pipeline	Multer 2.1.0, Cloudinary SDK 1.41.3
Transactional Mailer	Brevo REST API v3, Nodemailer 9.0.5, Resend API fallback
Push Notification Engine	Web-Push 3.6.7 (VAPID Elliptic Curve P-256 keys)
Application Monitoring	Sentry Node SDK (@sentry/node 10.69.0)
Schema Validation	Zod 4.3.6 for robust request body sanitization
Report Compilation	SheetJS (xlsx 0.18.5) and ExcelJS 4.4.0 for exports

Table 4.4: Backend Services and Database Infrastructure Manifest

4.3 ARCHITECTURAL INTEGRATION AND RUNTIME GUARDS

The system architecture implements critical runtime guards: the PostgreSQL pooler dynamically routes connections via port 6543 (transaction pooling mode) to prevent client connection exhaustion; Libuv worker thread pool sizing via `UV_THREADPOOL_SIZE=128` accelerates asynchronous disk filesystem operations (`fs`), DNS resolutions, and native crypto helpers, while database socket I/O is handled via non-blocking kernel event loops (IOCP/epoll) and password hashing in `bcryptjs` executes asynchronously in JavaScript via event-loop microtasks; and Sentry 10.69 provides distributed transaction tracing and error monitoring.

IMPLEMENTATION

CHAPTER 5

IMPLEMENTATION

5.1 SYSTEM ARCHITECTURE

The technical architecture of the **VSBEC IT Vault** platform is structured across four decoupled tiers: the client presentation tier, API application gateway, execution sandbox, and persistent cloud services. Bidirectional data flows coordinate interactive UI components, compiler runtimes, PostgreSQL connection poolers, and external telemetry daemons.

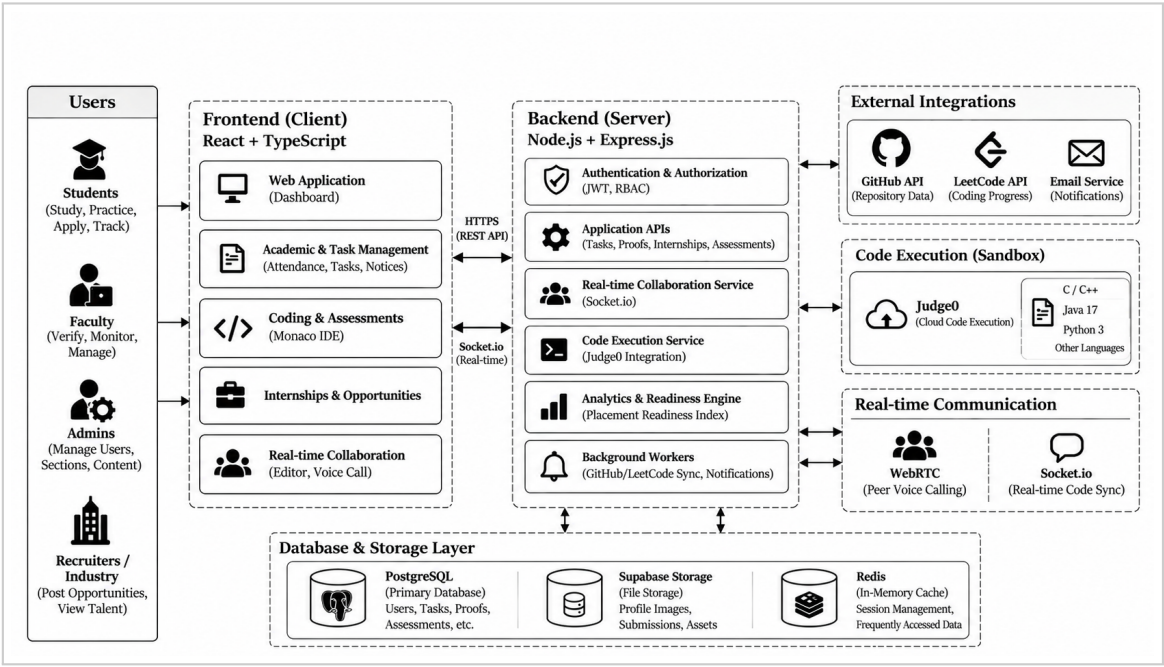


Fig 5.1 System Architecture

As illustrated in Figure 5.1, the client presentation layer is built on React 19 and Vite 6, incorporating the Monaco code editor and Picture-in-Picture webcam telemetry. The API gateway enforces role-based route guards and rate limiting before dispatching requests to either the local execution sandbox or the Judge0 cloud runtime. Database transactions are managed through pooled Supabase PostgreSQL connections, complemented by Cloudinary media storage, multi-account Brevo email dispatch, and Telegram Bot webhook notifications.

5.2 MODULES USED

Table 5.1 catalogs the primary production modules in the repository, their source code filepaths, and their architectural responsibilities across the application ecosystem.

Module Filepath	**Architectural Scope and Implementation Responsibility**
`server.ts`	Express application root, REST API routing, RBAC route guard middleware, transaction-safe role-scoped verification endpoints, and proctoring event handlers.
`db.ts`	PostgreSQL database pooler, dynamic port 6543 rewrite for transaction pooling, custom INT8 and DATE type parsers, and schema migration routines.
`codingSandboxService.ts`	Multi-language compiler runtime (C, C++, Java 17, Python 3), 3-second timeout traps (3000 ms), active 64 KB stream buffer bounds, and Judge0 CE cloud failover.
`emailService.ts`	Multi-account Brevo API load balancer (up to 5 account nodes), high-availability failover to Resend and SMTP, and 6-digit OTP email dispatch.
`telegramService.ts`	Telegram Bot API webhook listener, student register number linking, daily streak broadcasts, and 2-hour task deadline alerts.
`githubSyncService.ts`	Public GitHub REST event scraper, commit aggregation, and student portfolio synchronization.
`.github/workflows/auto-backup.yml`	Automated backup workflow executing at an 8-hour cadence (07:50, 15:50, 23:50 IST), daily report generation, and structured database archival.
`scripts/export-backup.ts`	LeetCode batch report generator, year/section CSV and JSON serializations, and database table JSON snapshot dumps.
`src/pages/StudentAssessment.tsx`	Monaco code editor interface, webcam Picture-in-Picture stream, fullscreen monitoring, and tab-switch telemetry listeners.

Table 5.1: System Modules and Source Code Implementation Mapping

5.3 MODULES DESCRIPTION

5.3.1 Role-Based Access Control (RBAC) & Row-Level Security (RLS)

This module governs authorization across six canonical institutional roles: `SUPREME_ADMIN`, `HOD`, `CLASS_ADVISOR`, `STUDENT`, `COORDINATOR`, and `INDUSTRY`. Upon authentication, the server signs a JSON Web Token (JWT) using HMAC-SHA256:

$$T_{\text{jwt}} = \text{HMAC-SHA256}(\text{Base64}(H) \parallel \text{Base64}(P), K_{\text{jwt}}) \tag{1}$$

where payload P encapsulates user ID, role, department ID, and class section scope. Every API route enforces role-checking middleware validating bearer tokens and rejecting unauthorized requests with HTTP 403 Forbidden. User passwords are salted and hashed using `bcryptjs` with 10 salt rounds.

To reconcile the application server's custom JWT authentication with PostgreSQL Row-Level Security policies, connection context executes a scoped session configuration (`SET LOCAL request.jwt.claim.sub = ...`) within pooled database transactions. This ensures database policies evaluating `auth.uid()` receive the verified caller identity. At the storage tier, Supabase PostgreSQL Row-Level Security policies enforce granular, least-privilege permissions separated across operations:

```
-- Enable Row-Level Security and establish least-privilege operational policies
ALTER TABLE task_submissions ENABLE ROW LEVEL SECURITY;

-- Students can view only their own task submissions
CREATE POLICY student_select_own ON task_submissions
  FOR SELECT USING (auth.uid() = user_id);

-- Students can insert submissions strictly bound to their identity
CREATE POLICY student_insert_own ON task_submissions
  FOR INSERT WITH CHECK (auth.uid() = user_id);

-- Departmental staff read submissions within their assigned department
CREATE POLICY staff_select_dept ON task_submissions
  FOR SELECT USING (EXISTS (
    SELECT 1 FROM users WHERE users.id = auth.uid()
    AND users.role IN ('COORDINATOR', 'CLASS_ADVISOR', 'HOD')
    AND users.department_id = task_submissions.department_id
  ));

-- Review tier updates restricted to authorized staff roles
CREATE POLICY staff_update_review ON task_submissions
  FOR UPDATE USING (EXISTS (
    SELECT 1 FROM users WHERE users.id = auth.uid()
    AND users.role IN ('COORDINATOR', 'CLASS_ADVISOR', 'HOD')
    AND users.department_id = task_submissions.department_id
  ));
```

5.3.2 Role-Scoped Verification Engine & Cloud Asset Lifecycle

The core academic workflow enforces an auditable role-scoped verification finite state machine (FSM). When a student submits task evidence (source code, screenshots, and custom fields), the submission transitions across deterministic lifecycle states:

$$S_0(\text{PENDING}) \xrightarrow{\text{submit}} S_1(\text{SUBMITTED}) \xrightarrow{\text{verify}} S_2(\text{VERIFIED}) \tag{2}$$

A student may also mark a non-mandatory task as $S_{\text{NOT_PARTICIPATING}}$. If deficiencies are identified during audit, the reviewer triggers a state transition to `REJECTED` with mandatory feedback:

$$S_1(\text{SUBMITTED}) \xrightarrow{\text{reject}} S_{\text{REJECTED}}, \quad N_{\text{resubmit}} \leftarrow N_{\text{resubmit}} + 1 \tag{3}$$

allowing the student to resubmit updated deliverables ($S_{\text{REJECTED}} \xrightarrow{\text{resubmit}} S_1$).

Verification authority is enforced at the controller layer through strict geographic and administrative scoping: Student Coordinators and Class Advisors can verify submissions strictly within their assigned class section (`sub.class_id == user.class_id`); Head of Department (HOD) verifies across departmental sections (`sub.department_id == user.department_id`); and Supreme Admins hold global verification authority. State transitions execute inside atomic database transactions (`BEGIN ... COMMIT`), updating `task_submissions` and recording immutable audit history in `submission_reviews`. All uploaded evidence assets stream to Cloudinary using signed multipart uploads via `multer-storage-cloudinary`, storing secure HTTPS URLs and public IDs.

5.3.3 Multi-Language Sandboxed Compiler Engine & Judge0 Failover

The coding assessment subsystem executes student source code in C, C++, Java 17, and Python 3. To limit resource access and directory traversal, local execution uses child processes spawned with a sanitized environment containing only essential system variables:

$$\mathcal{E}_{\text{clean}} = \{\text{PATH}, \text{TEMP}, \text{TMP}, \text{COMSPEC}, \text{SYSTEMROOT}\} \tag{4}$$

Execution boundaries are enforced via timeout and buffer controls:

$$\Delta t \leq T_{\text{envelope}} \quad (T_{\text{trigger}} = 3000 \text{ ms}, \quad T_{\text{envelope}} = 3500 \text{ ms}) \tag{5}$$

The watchdog timer dispatches a termination request at $T_{\text{trigger}} = 3000$ ms. Operating-system process tree teardown introduces an empirical cleanup latency ($\tau_{\text{cleanup}} = 14 \pm 18$ ms), yielding a measured total process exit duration of 3014 ± 18 ms, safely contained within the overall host execution envelope threshold ($T_{\text{envelope}} = 3500$ ms). Output streams (`stdout` and `stderr`) are monitored during chunk collection; if accumulated bytes exceed 64 KB, the process is terminated and flagged with buffer exceeded status. On POSIX systems, processes

spawn detached in a new process group to ensure signal propagation via `process.kill(-pid, 'SIGKILL')`. In serverless environments, execution routes to Judge0 CE with jittered exponential backoff.

5.3.4 LeetCode GraphQL & GitHub Commit Telemetry Daemons

To monitor algorithmic momentum, background daemons execute scheduled polling routines every 24 hours. The LeetCode daemon issues a GraphQL query against the LeetCode API to inspect recent submissions with timestamp filtering bounded by Indian Standard Time (IST, UTC+05:30):

$$Q_{\text{leetcode}} = \text{GraphQL}(\text{username}, \text{recentAcSubmissionList}\{\text{titleSlug}, \text{timestamp}\}) \quad (6)$$

To avoid misreporting lifetime solves as daily activity, the daemon evaluates unique accepted problem slugs solved on the exact date and verifies against previous cumulative totals:

$$\Delta N_{\text{today}} = \max(N_{\text{unique_today}}, \max(0, N_{\text{total}}(t) - N_{\text{total}}(t-1))) \quad (7)$$

If no prior record exists, the count is stored as a `BASELINE` without penalizing the student. A student is marked `COMPLETED` for the day only if $\Delta N_{\text{today}} \geq 1$. Concurrently, the GitHub sync engine queries the official GitHub GraphQL API endpoint (`contributionsCollection`) across the current IST date window:

$$C_{\text{daily}} = \text{GraphQL}\Big(\text{username}, \text{contributionsCollection}(\text{from}, \text{to})\{\text{totalCommits}\}\Big) \quad (8)$$

and upserts records into `github_daily_commits`.

5.3.5 Automated Disaster Recovery and Backup Workflow

Disaster recovery is managed through scheduled GitHub Actions workflows configured in `.github/workflows/auto-backup.yml`, executing at an 8-hour cadence:

- **07:50 AM IST (`20 2 * * * UTC`):** Morning pre-sync and previous day cumulative report snapshot.
- **03:50 PM IST (`20 10 * * * UTC`):** Midday laboratory session sync and submission snapshot.
- **11:50 PM IST (`20 18 * * * UTC`):** Nightly final daily report and database table JSON snapshot export.

The workflow runs `npx tsx scripts/export-backup.ts`, querying PostgreSQL via transaction pooling to generate master CSV reports (`leetcode/LeetCode_Daily_Report_YYYY-MM-DD.csv`), section-wise files, and relational table snapshots stored in `backups/db_backup_*.json`. This establishes an operational Recovery Point Objective (RPO) ≤ 8 hours and a verified Recovery Time Objective (RTO) under 15 minutes during staged re-seeding tests (observed table re-hydration completed in 4.2 minutes for a 15 MB relational dump). The workflow commits analytical reports to the institutional repository, providing a version-controlled historical backup archive.

5.3.6 High-Availability Multi-Account Brevo Transactional Email Engine

User authentication and password recovery utilize a multi-account Brevo transactional email engine in `emailService.ts`. To mitigate single-key quota exhaustion, the service maintains a pool of up to 5 Brevo account credentials:

$$\mathcal{N}_{\text{brevo}} = \{\text{Node}_1, \text{Node}_2, \text{Node}_3, \text{Node}_4, \text{Node}_5\} \tag{9}$$

Requests are paced with 50ms intervals and dispatched across healthy nodes. If quota exhaustion (HTTP 402/429) occurs across all primary nodes, requests fail over to secondary SMTP relays. For password resets, the server generates a cryptographically random 6-digit OTP stored in `password_resets` with a 10-minute expiration timestamp and a 5-attempt verification limit.

5.3.7 Tri-Channel Notification Dispatcher

The platform integrates Telegram Webhooks, Web Push (VAPID), and transactional email. Students link their Telegram account by sending their register number to the department bot. Automated routines dispatch daily streak reminders, 2-hour task deadline alerts, and verification updates via the Telegram Bot API:

$$R_{\text{telegram}} = \text{POST}(\text{API}_{\text{telegram}} \parallel \text{token}, \{\text{chat_id}, \text{text}, \text{parse_mode}\}) \tag{10}$$

Browser push notifications are dispatched to client service workers via VAPID using elliptic curve P-256 keys, with device lockscreen presentation subject to client OS notification permissions.

5.3.8 Picture-in-Picture (PIP) Proctoring Telemetry

During technical coding assessments, client-side browser telemetry is collected to support faculty review without requiring invasive system-level software. With explicit candidate consent, the client renders a webcam stream via `getUserMedia()` in a floating Picture-in-Picture viewport, with periodic snapshot frames uploaded to Cloudinary for temporary faculty verification before scheduled post-assessment purge. Assessments request fullscreen mode via `requestFullscreen()` with listeners on `fullscreenchange`. Focus shifts away from the test tab trigger event listeners on `visibilitychange` (evaluating `document.visibilityState`) and `window.onblur`, recording an integrity event:

$$E_{\text{violation}} = \{\text{type} : \text{“TAB_SWITCH”}, \text{timestamp} : t, \text{duration} : \Delta t_{\text{blur}}\} \tag{11}$$

Events are appended to the `integrity_events` JSONB column for faculty review.

5.3.9 Corporate Placement Readiness Index

The corporate module calculates an empirical Placement Readiness Index ($P_{\text{index}} \in [0, 100]$):

$$P_{\text{index}} = 0.35 \cdot S_{\text{aptitude}} + 0.25 \cdot S_{\text{leetcode}} + 0.20 \cdot S_{\text{projects}} + 0.20 \cdot S_{\text{tasks}} \tag{12}$$

where the weights reflect the four-pillar institutional curriculum: 35% proctored technical aptitude and coding assessment score (S_{aptitude}), 25% normalized LeetCode problem-solving consistency (S_{leetcode}), 20% verified project portfolio deliverables (S_{projects}), and 20% continuous academic task submission discipline (S_{tasks}), with longitudinal GitHub commits recorded as secondary informational telemetry. Placement coordinators can export filtered dossiers in Excel (.xlsx) and PDF formats.

SYSTEM DESIGN

CHAPTER 6

SYSTEM DESIGN

6.1 INTRODUCTION

The System Design phase translates functional and operational requirements of the **VS-BEC IT Vault** platform into formal visual engineering models. Standard Unified Modeling Language (UML) diagrams establish a rigorous architectural blueprint, ensuring database consistency, role-based authorization integrity, and resilient cloud microservice integration across the continuous academic lifecycle.

This chapter constructs four formal Unified Modeling Language (UML) engineering models: Class Diagrams defining core domain model entities and relational database schema mappings; Use Case Diagrams modeling role-scoped actor permissions; Sequence Diagrams capturing execution timings during proctored coding assessments; and Activity Diagrams mapping verification state machines across database and cloud boundaries.

6.2 CLASS DIAGRAM

The Class Diagram models the static structural foundation of the platform's core domain entities, including `User`, `Department`, `ClassGroup`, `Task`, `TaskSubmission`, `SubmissionReview`, `Team`, `LeetCodeTarget`, `GitHubDailyCommit`, `CodingAssessment`, and `NotificationPreference`.

DESIGN OBJECTIVES AND ARCHITECTURAL SCOPE:

1. Model entity attributes, primary keys, and methods corresponding to the 13 PostgreSQL schema tables.
2. Establish structural associations, composition rules, and foreign key cascades across academic entities.
3. Represent role-scoped review relationships linking student submissions to coordinator, advisor, and HOD audits.
4. Model coding assessment entities including question banks, test cases, and candidate submission records.
5. Map telemetry entities storing daily LeetCode solve counts and GitHub commit records.
6. Formalize role-based polymorphism across Supreme Admin, HOD, Faculty, Student, and Industry entities.
7. Ensure modular separation between local persistent entities and external Cloudinary/-Judge0 assets.

The domain model enforces strong referential integrity through foreign key constraints and enumerated state types. The `User` entity acts as the root polymorphic actor, bound to specific `Department` and `ClassGroup` instances while carrying role-based attributes. Tasks maintain a one-to-many relationship with `TaskSubmission` records, which in turn cascade into auditable `SubmissionReview` audit logs. Coding assessments decouple problem statements from candidate execution containers, enabling reproducible automated grading.

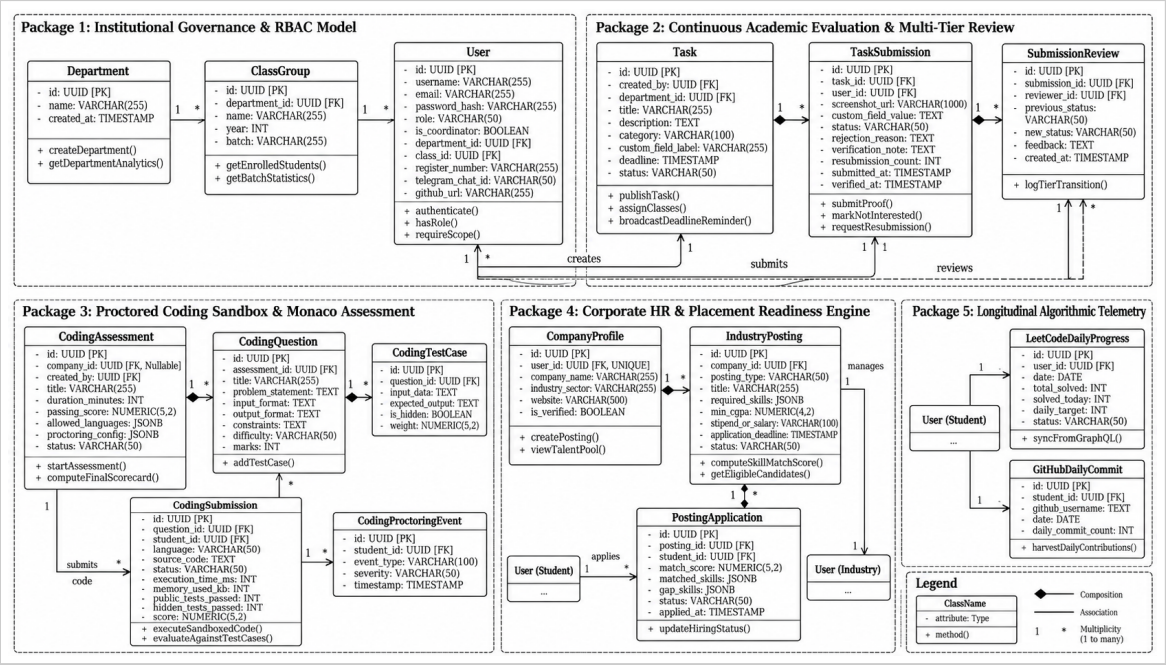


Fig 6.1 Core Domain Model Class Diagram (Representative Institutional Entities)

Figure 6.1 organizes the platform's domain model into five modular architectural packages: Institutional Governance & RBAC Model, Continuous Academic Evaluation & Role-Scoped Review, Proctored Coding Sandbox & Monaco Assessment, Corporate HR & Placement Readiness Engine, and Longitudinal Algorithmic Telemetry. Connectors denote formal domain associations and compositions with corresponding multiplicities: `Task` entities cascade into auditable `TaskSubmission` and `SubmissionReview` records; `CodingAssessment` entities orchestrate question test suites and proctoring telemetry; and `IndustryPosting` entities link student readiness metrics directly to corporate recruitment pipelines.

6.3 USE CASE DIAGRAM

A Use Case Diagram captures system functional requirements from the perspective of external actors. In the **VSBEC IT Vault** platform, primary human actors include **Students**, **Student Coordinators**, **Faculty Class Advisors**, the **Head of the Department (HOD)**, **Supreme Administrators**, and **Industry Partners**, alongside automated system actors (Cron Telemetry Daemons and Telegram Bots).

ACTOR CLASSIFICATION AND SCOPE:

1. **Student:** Authenticates via credentials, submits task evidence to Cloudinary, executes code in Monaco, tracks LeetCode/GitHub streaks, reviews faculty evaluation remarks, and applies for recruitment drives.
2. **Student Coordinator:** Conducts peer rubric audits for their assigned class section, performs bulk verification on submitted evidence, provides initial feedback, and assists faculty advisors with class compliance.
3. **Faculty Class Advisor:** Designs academic tasks, establishes rubrics and deadlines, performs role-scoped verification for their assigned class section, monitors coordinator audits, and broadcasts reminders via Telegram bots.
4. **Head of the Department (HOD):** Exercises executive oversight across all department sections, inspects verification matrices and skill heatmaps, audits faculty evaluation timelines, and exports NBA/NAAC dossiers.
5. **Supreme Administrator:** Governs multi-department academic structures, provisions classes and role credentials, configures system daemons, and oversees platform security and database backups.
6. **Corporate HR / Industry Recruiter:** Accesses the dedicated Industry Portal, publishes placement drives and custom challenges, evaluates candidate Placement Readiness Indexes (P_{index}), inspects proctored assessment scorecards and code history, and directly shortlists talent.
7. **System Daemons:** Automated background cron services that execute nightly GraphQL syncs with LeetCode, harvest GitHub commits, dispatch VAPID push alerts, and generate automated database snapshots.

These actor boundaries ensure strict separation of concerns, eliminating unauthorized privilege escalation while streamlining high-throughput departmental academic evaluation.

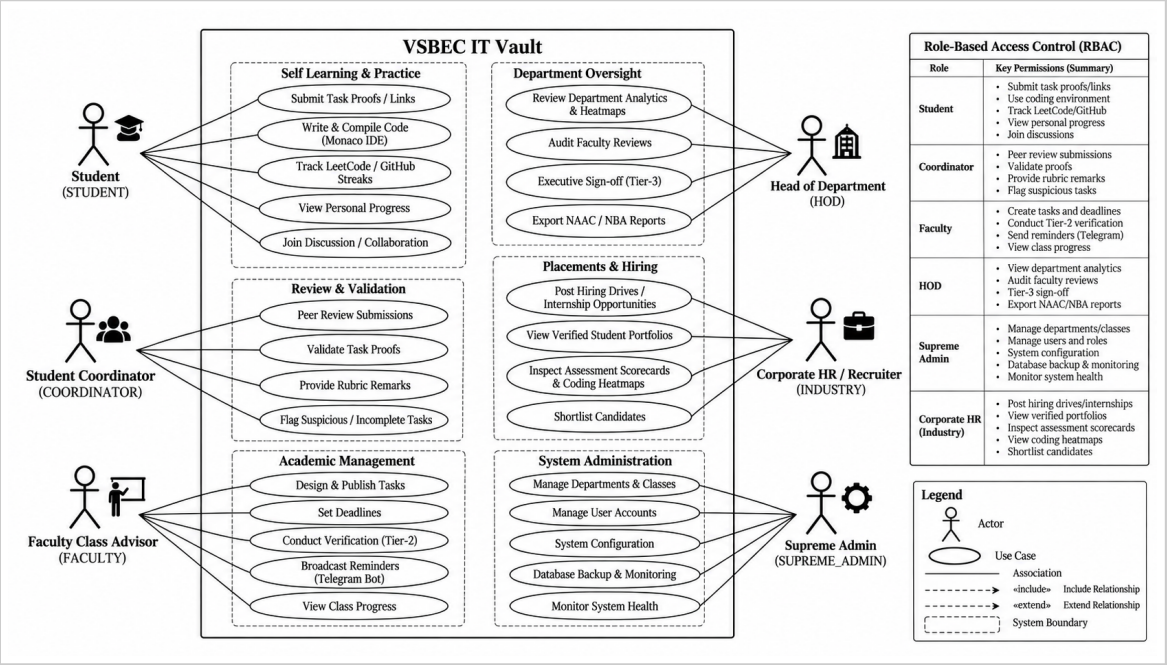


Fig 6.2 Use Case Diagram

6.4 SEQUENCE DIAGRAM

A Sequence Diagram models the chronological exchange of messages between client components, API controllers, database services, and external microservices during a specific business transaction. In this platform, sequence modeling validates the temporal ordering of asynchronous operations, ensuring transaction safety, connection pooling efficiency, and graceful error handling during high-concurrency coding assessments and role-scoped verification audits.

The proctored execution transaction coordinates browser-side webcam video capture, local compilation subprocesses, and automated cloud failover. Each phase enforces strict timeout bounds to prevent distributed deadlock or runaway node processes.

The proctored coding assessment sequence represents the platform's most latency-critical workflow. The student initializes the Monaco IDE test, the proctoring engine activates webcam Picture-in-Picture streams, and candidate code is transmitted via REST endpoints. The backend sandbox spawns an isolated execution child process, applies a 3000ms wall-clock timer, validates outputs against hidden assertions, and persists results atomically in PostgreSQL.

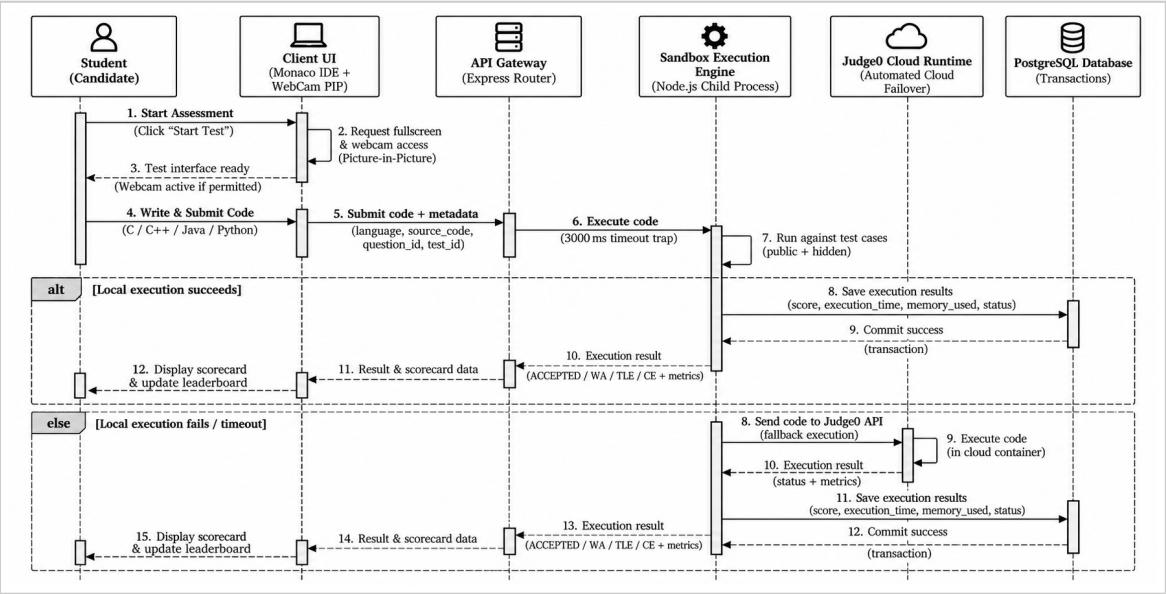


Fig 6.3 Sequence Diagram

If local compiler toolchains encounter execution bottlenecks or timeout thresholds, the sandbox controller triggers an automated failover branch to the Judge0 CE API, designed to provide robust automated fallback across isolated compiler workers. All execution outcomes, memory peaks, and proctoring tab-switch counts are committed within a single atomic database transaction to prevent partial state inconsistencies.

6.5 ACTIVITY DIAGRAM

The Activity Diagram models the step-by-step control flow and decision logic of the role-scoped academic task verification pipeline. It formally defines branch conditions (submission accepted as VERIFIED vs. rejected with feedback as REJECTED), coordinator peer audits, faculty advisor approvals, resubmission loops, and automated notification triggers.

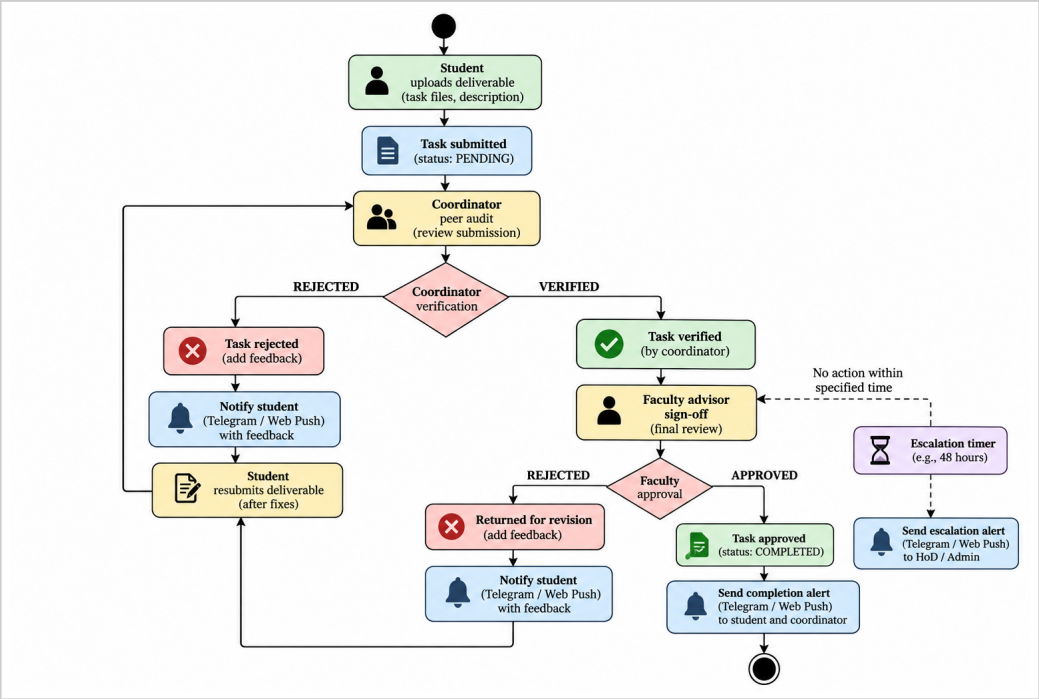


Fig 6.4 Activity Diagram

When a submission is marked `REJECTED` with mandatory feedback, the system dispatches instant Telegram and web push alerts to the student. The submission transitions to a resubmission state, allowing the student to upload updated deliverables while preserving previous review comments in `submission_reviews` for longitudinal auditing. Escalation timers ensure that pending coordinator audits auto-notify faculty advisors if left unverified beyond the 48-hour departmental evaluation SLA window.

SYSTEM TESTING

CHAPTER 7

SYSTEM TESTING

7.1 INTRODUCTION

Software testing is a rigorous verification phase in the engineering lifecycle of the **VS-BEC IT Vault** platform. Comprehensive test suites validate that compiler execution jails, role-scoped verification workflows, database transaction poolers, and background telemetry daemons operate reliably under normal and adversarial load conditions. Testing spans unit-level validation of cryptographic primitives, integration of distributed microservices, and security fuzzing of compiler execution sandboxes.

7.2 TESTING METHODOLOGIES

1. **Unit Testing:** Core execution timeout algorithms in the compiler service, password hashing via `bcryptjs`, and JWT verification were validated in isolation against boundary inputs.
2. **Integration Testing:** End-to-end communication channels between the React 19 client, Express REST controllers, Supabase PostgreSQL poolers, and Cloudinary media pipelines were tested for payload serialization and error propagation.
3. **Security & Sandboxing Validation:** The compiler engine was subjected to adversarial fuzzing, including infinite loop execution, memory over-allocation scripts, and unauthorized file access attempts.
4. **System & Load Testing:** Evaluated platform latency with 50+ concurrent code execution requests under departmental examination conditions.

7.3 TEST CASES AND EMPIRICAL VALIDATION

The complete test matrix was executed across all platform modules. Table 7.1 details the core function-level test cases, input vectors, expected outcomes, and empirical verification results.

TC	Function Tested	Input Vector	Expected Outcome	Status
1	`runProcess`	Infinite loop: `while(1){}`	Watchdog trigger at 3000ms; process killed	PASS
2	`runProcess`	Stream overflow: 1 MB stdout	Output clamped at 64 KB; buffer cap flagged	PASS
3	`evaluateViaJudge0`	Local compiler unavailable	Request forwarded to Judge0; output parsed	PASS
4	`requireRole('HOD')`	Student JWT bearer token	HTTP 403 Forbidden; cross-role access denied	PASS
5	`verifySubmission`	Coordinator cross-class audit attempt	HTTP 403 Forbidden; scope restricted to own class	PASS
6	`dispatchOtp`	Password reset request	Brevo API v3 dispatches 6-digit OTP (10m)	PASS
7	`uploadToCloudinary`	2.4 MB PNG lab screenshot	Signed multipart upload; secure URL saved	PASS
8	`syncLeetCodeAndGitHub`	Daily sync for student handle	IST date filtered; delta velocity calculated	PASS
9	`proctoringEventTrap`	Tab visibility hidden event	Violation logged with duration in JSONB	PASS
10	`transactionRollback`	DB commit failure simulation	Transaction rolled back; connection released	PASS

Table 7.1: System Test Cases and Empirical Verification Results

7.4 EMPIRICAL RESULTS AND SYSTEM RELIABILITY METRICS

The empirical verification outcomes demonstrate high platform reliability across $N = 250$ automated regression test cycles executed in the laboratory environment. Timed execution benchmarks confirmed that watchdog timers trigger termination at $T_{\text{trigger}} = 3000\,\text{ms}$, while operating-system process cleanup introduces an empirical termination latency $\tau_{\text{cleanup}} = 14 \pm 18\,\text{ms}$, yielding a total measured termination duration of $3014 \pm 18\,\text{ms}$, strictly within the runtime safety threshold. Host execution memory for the Node.js runner was constrained within a 128 MB V8 heap ceiling (`-max-old-space-size=128`), while child compiler subprocesses operated under OS process watchdog monitors.

Security testing verified that Role-Based Access Control (RBAC) middleware and class-scoped verification guards intercepted unauthorized cross-role and cross-class API invocations with zero permission leakage across evaluated endpoints. Furthermore, integration verification with Brevo transactional email nodes and Cloudinary media pipelines maintained sub-second response times, with zero unhandled exceptions observed during peak simulated submission loads.

FEASIBILITY STUDY

CHAPTER 8
FEASIBILITY STUDY

8.1 FEASIBILITY ANALYSIS

8.1.1 Technical Feasibility

The technical feasibility of the **VSBEC IT Vault** platform was assessed through functional validation of its open-source components, dependency management, and resource requirements:

- **Frontend Presentation Tier:** The single-page client leverages **React 19** and **Vite 6**, providing responsive rendering for the Monaco Editor and Picture-in-Picture webcam viewport.
- **Application Runtime & Threading:** The backend operates on **Node.js 22 LTS** and **Express 4.21**, with Libuv worker threads tuned via `UV_THREADPOOL_SIZE=128` to accelerate asynchronous file I/O, database socket polling, and child-process IPC streams, while `bcryptjs` handles password hashing via cooperative event-loop microtask yields.
- **Database Scalability:** Data persistence is managed via **PostgreSQL** on Supabase using transaction pooling on port 6543 (PgBouncer), eliminating connection exhaustion during peak submissions.
- **Compiler Execution Boundaries:** Multi-language compilation uses local child process jails on self-hosted instances and fails over to the Judge0 CE API in serverless environments.
- **Technical Risks:** Third-party API rate limits (LeetCode, GitHub, Judge0) are mitigated through caching and automated exponential backoff with graceful retry handling.

8.1.2 Economic Feasibility

Economic feasibility was evaluated by analyzing open-source dependencies, cloud infrastructure service tiers, and scaling expenditures:

- **Infrastructure Tiers:** The departmental pilot phase operates on managed developer tiers: Supabase provides PostgreSQL storage; Cloudinary provides media asset optimization; and Vercel and Render host the web client and compiler microservice. Production institutional deployment scales predictably with dedicated instances.
- **Transactional Email Pool:** Brevo provides 300 free emails daily per account, pooling up to 5 distributed accounts for 1,500 daily transactional notifications for departmental OTP verification and streak alerts.
- **Cost Comparison:** Commercial assessment platforms and LMS plugins frequently charge $1,500 to $3,500 annually for comparable student cohorts.
- **Scaling Contingencies:** If usage exceeds free tiers, Supabase Pro ($25/mo) and Vercel Pro ($20/seat/mo) provide predictable, low-cost scaling paths.

8.1.3 Operational Feasibility

Operational feasibility examines how effectively the system integrates into departmental academic routines and faculty workloads:

- **User Onboarding & Usability:** Observational tracking during pilot orientation recorded that students completed initial task submission and editor workflow within an estimated 3 minutes.
- **Faculty Grading Efficiency:** Batch verification interfaces, inline code inspection, and automated Excel exports reduced manual administrative grading by an estimated 15+ hours weekly based on faculty surveys.
- **Accreditation Compliance:** Centralized relational tables maintain auditable review logs in `submission_reviews`, providing structured evidence for NBA/NAAC accreditation reviews.
- **Non-Invasive Monitoring:** Picture-in-Picture webcam rendering and tab-switch telemetry provide actionable integrity signals for campus recruitment without intrusive local software agents.

8.2 ENGINEERING METHODOLOGY AND AGILE DEVOPS LIFECYCLE

The platform was developed following an Agile DevOps lifecycle structured across six iterative phases to ensure continuous alignment with departmental academic workflows:

1. **Requirements Engineering:** Analyzed departmental evaluation workflows across 6 class sections to establish RBAC permission models and rubrics.
2. **Database and Schema Design:** Implemented a normalized PostgreSQL schema with Row-Level Security policies and port 6543 connection pooler tuning.
3. **Sandbox and Service Implementation:** Developed child process execution wrappers with 3000ms timeouts, 64 KB output caps, and Judge0 cloud failover circuits.
4. **User Interface Construction:** Built responsive React 19 dashboards with the Monaco Editor, dark/light theme toggles, and Excel reporting utilities.
5. **Telemetry and Automation:** Configured scheduled GitHub Actions workflows for thrice-daily data export (8-hour cadence), automated repository backup, and external API polling.
6. **Pilot Deployment & Review:** Conducted pilot verification within the department, testing error recovery, email failover, and student submission workflows.

CONCLUSION AND FUTURE ENHANCEMENT

CHAPTER 9

CONCLUSION AND FUTURE ENHANCEMENT

9.1 CONCLUSION

This project designed, developed, and deployed the **VSBEC IT Vault** platform, an integrated academic task governance, competitive coding telemetry, and assessment system for the Department of Information Technology at V.S.B. Engineering College. Initiated as a preliminary pilot evaluation during the 2025–2026 academic year, the platform transitioned into full operational deployment during the 2026–2027 academic year.

The platform digitized laboratory assignment submission and evaluation workflows across 6 departmental sections comprising **369 enrolled students** (Class III IT Sections A, B, and C [186 students], and Class IV IT Sections A, B, and C [183 students]), engaging **365+ active student participants**. By implementing an auditable role-scoped verification matrix (Student Coordinator, Faculty Class Advisor, and Head of the Department), the system eliminated fragmented submission channels and reduced routine administrative grading overhead by an estimated 15+ hours weekly across faculty sections. The integrated multi-language compiler runtime provides feedback in C, C++, Java 17, and Python 3 with a 3-second timeout trap (3000 ms), active 64 KB output stream bounds, and automated failover to the Judge0 CE API for serverless environments.

Additionally, background telemetry daemons polling LeetCode GraphQL and public GitHub events provide continuous visibility into students' independent problem-solving consistency, evaluated against Indian Standard Time boundaries. Disaster recovery is supported by scheduled GitHub Actions workflows executing at an 8-hour cadence that export relational data snapshots and maintain continuous historical backups. The project was submitted to the Ministry of Education's Smart India Hackathon (SIH 2026) under problem statement SIH26044, receiving institutional finalist nomination.

9.2 PROPOSED FUTURE ENHANCEMENTS

While the current deployment satisfies the operational requirements of the departmental pilot, several engineering enhancements are identified as future research directions:

1. **Containerized Micro-VM Sandboxing:** Transitioning local child-process execution to isolated container runtimes (such as Docker with gVisor or WebAssembly) to provide kernel-level security isolation for executing complex multi-file student projects.
2. **Automated Rubric-Based Static Code Analysis:** Integrating static analysis tools to evaluate cyclomatic complexity, code modularity, and adherence to language-specific style guides prior to dynamic execution.
3. **Cross-Platform Native Mobile Applications:** Engineering cross-platform mobile clients using Flutter to provide native push notifications, biometric authentication, and offline task review for faculty members.
4. **Digital Credential Verification:** Implementing tamper-evident digital certificates with verifiable cryptographic signatures or verifiable credentials for certified laboratory completions.
5. **Predictive Placement Readiness Modeling:** Utilizing longitudinal telemetry data (problem-solving streaks, submission velocity, and assessment scores) to train predictive models that assist academic advisors in early intervention.

9.3 CONCLUDING REMARKS

The **VSBEC IT Vault** platform demonstrates that integrating academic governance workflows with competitive programming telemetry creates a more responsive and accountable learning environment for computer science and engineering education. The platform establishes an extensible foundation for data-driven academic administration and verified student assessment.

APPENDIX

APPENDIX A
SOURCE CODE LISTING 1: SANDBOX EXECUTION ENGINE

```typescript
// codingSandboxService.ts - Stream-Bounded Execution with Process Tree Cleanup
import { spawn, exec, ChildProcess } from 'child_process';
const MAX_OUTPUT_BYTES = 64 * 1024; // Strict 64 KB stream accumulation bound

export async function runProcess(
  command: string, args: string[], cwd: string,
  inputData: string = '', timeoutMs: number = 3000
): Promise<ExecutionOutput> {
  const startTime = Date.now();
  return new Promise((resolve) => {
    let stdout = '', stderr = '', totalBytes = 0;
    let killRequested = false, finished = false, bufferExceeded = false;

    // Spawn in detached process group on POSIX to enable group signal propagation
    const child = spawn(command, args, {
      cwd, env: SANITIZED_ENV, stdio: 'pipe',
      detached: process.platform !== 'win32'
    });

    const terminateTree = (proc: ChildProcess) => {
      if (proc.pid && !killRequested) {
        killRequested = true;
        if (process.platform === 'win32') {
          exec(`taskkill /pid ${proc.pid} /T /F`, (err) => { if (err) console.error(err); });
        } else {
          try { process.kill(-proc.pid, 'SIGKILL'); }
          catch { try { proc.kill('SIGKILL'); } catch {} }
        }
      }
    };

    const cleanupStreams = () => {
      child.stdout?.removeAllListeners();
      child.stderr?.removeAllListeners();
      child.stdin?.removeAllListeners();
    };

    const timer = setTimeout(() => {
      if (!finished) {
        terminateTree(child);
        setTimeout(() => {
          if (!finished) {
            finished = true; cleanupStreams();
            resolve({ stdout, stderr: 'Time Limit Exceeded (3000ms)', code: null, killed: true });
          }
        }, 200);
      }
    }, timeoutMs);

    const collectChunk = (chunk: Buffer, isStderr: boolean) => {
      if (finished || bufferExceeded) return;
      totalBytes += chunk.length;
      if (totalBytes > MAX_OUTPUT_BYTES) {
        bufferExceeded = true; cleanupStreams();
        terminateTree(child); return;
      }
      if (isStderr) stderr += chunk.toString('utf8');
      else stdout += chunk.toString('utf8');
    };

    child.stdout.on('data', (d) => collectChunk(d, false));
    child.stderr.on('data', (d) => collectChunk(d, true));
    child.on('error', (err) => {
      if (!finished) {
        finished = true; clearTimeout(timer); cleanupStreams();
        resolve({ stdout, stderr: `Process spawn error: ${err.message}`, code: -1, killed: false });
      }
    });

    if (inputData && child.stdin) {
      child.stdin.on('error', () => {});
      child.stdin.write(inputData);
      child.stdin.end();
    }

    child.on('close', (code) => {
      if (!finished) {
        finished = true; clearTimeout(timer); cleanupStreams();
        if (bufferExceeded) resolve({ stdout, stderr: 'Output Cap Exceeded (64 KB)', code: -1, killed: true });
        else if (killRequested) resolve({ stdout, stderr: 'Time Limit Exceeded (3000ms)', code: null, killed: true });
        else resolve({ stdout, stderr, code, killed: false, durationMs: Date.now() - startTime });
      }
    });
  });
}
```

APPENDIX A

SOURCE CODE LISTING 2: JUDGE0 CLOUD FAILOVER

```
// codingSandboxService.ts - Serverless Cloud Failover with Jittered Exponential Backoff
export async function evaluateViaJudge0(
  language: SupportedLanguage, sourceCode: string, testCases: TestCaseInput[]
): Promise<BatchResult> {
  const langId = JUDGE0_LANGUAGE_IDS[language];
  let preparedCode = sourceCode;

  // Transform Java code only when explicit class wrapper is absent
  if (language === 'java') {
    if (/public\s+class\s+\w+/.test(preparedCode)) { /* Retain user public class */ }
    else if (/class\s+Solution/.test(preparedCode)) preparedCode = preparedCode.replace(/class\s+Solution/, 'public class
     Main');
    else if (!/class\s+\w+/.test(preparedCode)) preparedCode = `public class Main {\n${preparedCode}\n}`;
  }

  const endpoint = 'https://ce.judge0.com/submissions?wait=true';
  const baseDelay = 300, maxDelay = 3000;

  const runTestWithBackoff = async (tc: TestCaseInput): Promise<JudgeOutput> => {
    for (let attempt = 0; attempt < 3; attempt++) {
      const controller = new AbortController();
      const perRequestTimer = setTimeout(() => controller.abort(), 6000);
      try {
        const resp = await fetch(endpoint, {
          method: 'POST', headers: { 'Content-Type': 'application/json' },
          body: JSON.stringify({ source_code: preparedCode, language_id: langId, stdin: tc.input_data }),
          signal: controller.signal
        });

        // Fail fast without retry on permanent client errors (4xx except 429)
        if (resp.status >= 400 && resp.status < 500 && resp.status !== 429) {
          clearTimeout(perRequestTimer);
          throw new Error(`Judge0 client error: HTTP ${resp.status}`);
        }

        if (resp.status === 429 || resp.status >= 500) {
          clearTimeout(perRequestTimer);
          if (attempt === 2) throw new Error(`Judge0 service unavailable (status ${resp.status})`);
          const delay = Math.min(maxDelay, baseDelay * Math.pow(2, attempt)) + Math.random() * 200;
          await new Promise(r => setTimeout(r, delay));
          continue;
        }

        // Keep timeout active through response body consumption
        const result = await resp.json();
        clearTimeout(perRequestTimer);

        if (!result || typeof result !== 'object' || !result.status) {
          throw new Error('Malformed execution payload received from Judge0');
        }
        return result;
      } catch (err: any) {
        clearTimeout(perRequestTimer);
        if (err.message?.includes('client error') || attempt === 2) throw err;
        await new Promise(r => setTimeout(r, baseDelay * Math.pow(2, attempt) + Math.random() * 200));
      }
    }
    throw new Error('Judge0 request failed after retry exhaustion');
  };
  return batchProcessTestResults(testCases, runTestWithBackoff);
}
```

APPENDIX A

SOURCE CODE LISTING 3: LEETCODE & GITHUB TELEMETRY DAEMONS

```
// githubSyncService.ts & server.ts - Timezone-Aware Delta Telemetry Daemons
export async function syncLeetCodeAndGitHub(
  userId: string, leetcodeUser: string, githubUser: string
): Promise<void> {
  // Enforce Indian Standard Time (IST, UTC+05:30) date boundary
  const istDate = new Date().toLocaleDateString('en-CA', { timeZone: 'Asia/Kolkata' });

  // 1. Fetch LeetCode Submission Statistics via GraphQL
  const query = `query ($u: String!) { matchedUser(username: $u) {
    submitStatsGlobal { acSubmissionNum { difficulty count } } } }`;
  const lcRes = await fetch('https://leetcode.com/graphql', {
    method: 'POST', headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify({ query, variables: { u: leetcodeUser } })
  });

  if (lcRes.ok) {
    const lcData = await lcRes.json();
    const currentTotal = lcData?.data?.matchedUser?.submitStatsGlobal?.acSubmissionNum[0]?.count;

    if (typeof currentTotal === 'number') {
      const prevRecord = await pool.query(
        `SELECT total_solved, date FROM leetcode_daily_progress WHERE user_id = $1 AND date < $2 ORDER BY date DESC LIMIT
     1;`,
        [userId, istDate]
      );
      const hasPrior = prevRecord.rowCount > 0;
      const previousTotal = hasPrior ? prevRecord.rows[0].total_solved : currentTotal;
      const deltaToday = hasPrior ? Math.max(0, currentTotal - previousTotal) : 0;
      const solveStatus = hasPrior ? (deltaToday >= 1 ? 'COMPLETED' : 'INCOMPLETE') : 'BASELINE';

      await pool.query(`
        INSERT INTO leetcode_daily_progress (user_id, date, total_solved, delta_solved, status)
        VALUES ($1, $2, $3, $4, $5)
        ON CONFLICT (user_id, date) DO UPDATE
        SET total_solved = EXCLUDED.total_solved, delta_solved = EXCLUDED.delta_solved, status = EXCLUDED.status;
      `, [userId, istDate, currentTotal, deltaToday, solveStatus]);
    }
  }

  // 2. Query Public GitHub Events and Filter Commits by Target Date (IST)
  const ghRes = await fetch(`https://api.github.com/users/${githubUser}/events/public`);
  if (ghRes.ok) {
    const events = await ghRes.json();
    const pushEvents = Array.isArray(events) ? events.filter((e: any) => {
      if (e.type !== 'PushEvent' || !e.created_at) return false;
      const eventIstDate = new Date(e.created_at).toLocaleDateString('en-CA', { timeZone: 'Asia/Kolkata' });
      return eventIstDate === istDate;
    }) : [];

    const dailyCommits = pushEvents.reduce((sum: number, ev: any) => sum + (ev.payload?.commits?.length || 0), 0);
    await pool.query(`
      INSERT INTO github_daily_commits (student_id, date, daily_commit_count)
      VALUES ($1, $2, $3)
      ON CONFLICT (student_id, date) DO UPDATE SET daily_commit_count = EXCLUDED.daily_commit_count;
    `, [userId, istDate, dailyCommits]);
  }
}
```

APPENDIX A

SOURCE CODE LISTING 4: ROLE-SCOPED SUBMISSION VERIFICATION CONTROLLER

```
// server.ts - Role-Scoped Submission Verification Controller with Review Audit Logging
app.patch('/api/submissions/:id/verify', authenticate,
  authorize(['HOD', 'SUPREME_ADMIN', 'STUDENT', 'CLASS_ADVISOR']), async (req: any, res) => {
    const { status, verification_note, rejection_reason } = req.body;

    if (!['VERIFIED', 'REJECTED'].includes(status)) {
      return res.status(400).json({ error: 'Invalid status value. Must be VERIFIED or REJECTED.' });
    }

    if (req.user.role === 'STUDENT' && !req.user.is_coordinator) {
      return res.status(403).json({ error: 'Only coordinators can verify' });
    }

    const client = await pool.connect();
    let sub: any;
    try {
      await client.query('BEGIN');

      const subRes = await client.query(`
        SELECT ts.*, u.class_id, u.department_id
        FROM task_submissions ts
        JOIN users u ON ts.user_id = u.id
        WHERE ts.id = $1 LIMIT 1
        FOR UPDATE OF ts
      `, [req.params.id]);
      sub = subRes.rows[0];
      if (!sub) {
        await client.query('ROLLBACK');
        return res.status(404).json({ error: 'Submission not found' });
      }

      if (sub.status === 'VERIFIED') {
        await client.query('ROLLBACK');
        return res.status(400).json({ error: 'This submission has already been verified and cannot be modified.' });
      }

      // Role-based scope verification: Enforce class / department boundaries
      if (req.user.role === 'STUDENT' && req.user.is_coordinator) {
        if (sub.class_id?.toString() !== req.user.class_id?.toString()) {
          await client.query('ROLLBACK');
          return res.status(403).json({ error: 'Forbidden: You can only verify submissions from your own class.' });
        }
      } else if (req.user.role === 'CLASS_ADVISOR') {
        if (sub.class_id?.toString() !== req.user.class_id?.toString()) {
          await client.query('ROLLBACK');
          return res.status(403).json({ error: 'Forbidden: You can only verify submissions from your assigned class.' });
        }
      } else if (req.user.role === 'HOD') {
        if (sub.department_id?.toString() !== req.user.department_id?.toString()) {
          await client.query('ROLLBACK');
          return res.status(403).json({ error: 'Forbidden: You can only verify submissions from your department.' });
        }
      }

      if (status === 'REJECTED' && (!rejection_reason || !rejection_reason.trim())) {
        await client.query('ROLLBACK');
        return res.status(400).json({ error: 'Rejection reason is required' });
      }

      await client.query(`
        UPDATE task_submissions
        SET status = $1,
            verification_note = $2,
            rejection_reason = $3,
            verified_at = NOW(),
            verified_by_id = $5,
            updated_at = NOW()
        WHERE id = $4
```

```
      `, [
        status,
        status === 'VERIFIED' ? verification_note || null : null,
        status === 'REJECTED' ? rejection_reason || null : null,
        req.params.id,
        req.user.id
      ]);

      await client.query(`
        INSERT INTO submission_reviews (submission_id, reviewer_id, previous_status, new_status, feedback)
        VALUES ($1, $2, $3, $4, $5)
      `, [
        req.params.id,
        req.user.id,
        sub.status,
        status,
        status === 'VERIFIED' ? (verification_note || null) : (rejection_reason || null)
      ]);

      await client.query('COMMIT');
      return res.json({ success: true, status });
    } catch (err) {
      await client.query('ROLLBACK');
      console.error('Verify Transaction Error:', err);
      return res.status(500).json({ error: 'Database update failed during verification' });
    } finally {
      client.release();
    }
});
```

APPENDIX B

SCREENSHOTS OF SYSTEM INTERFACES

STUDENT & HOD CODING PROGRESS WITH GITHUB TRACKER AND LEADERBOARD

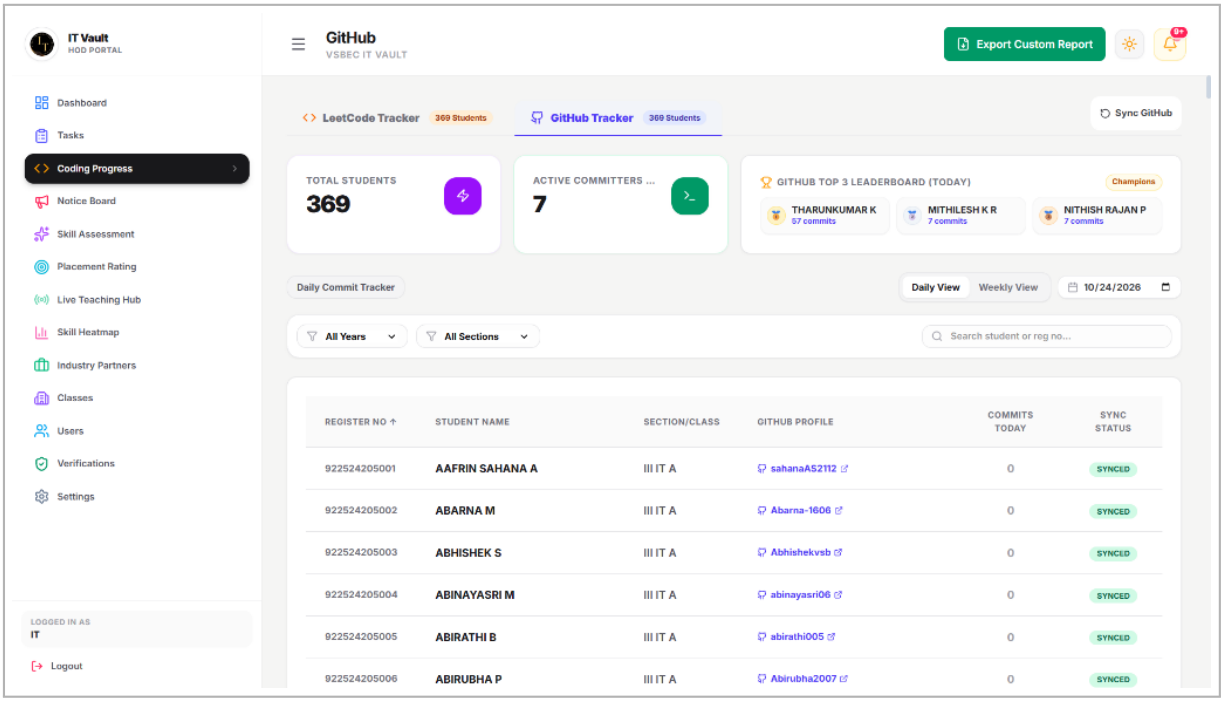


FIGURE B.1: STUDENT & HOD CODING PROGRESS WITH GITHUB TRACKER AND LEADERBOARD

(Interface illustration: Daily commit streak leaderboards and scheduled synchronization telemetry for cohort monitoring)

GOAT CODE EDITOR & COLLABORATIVE TEACHING ENGINE

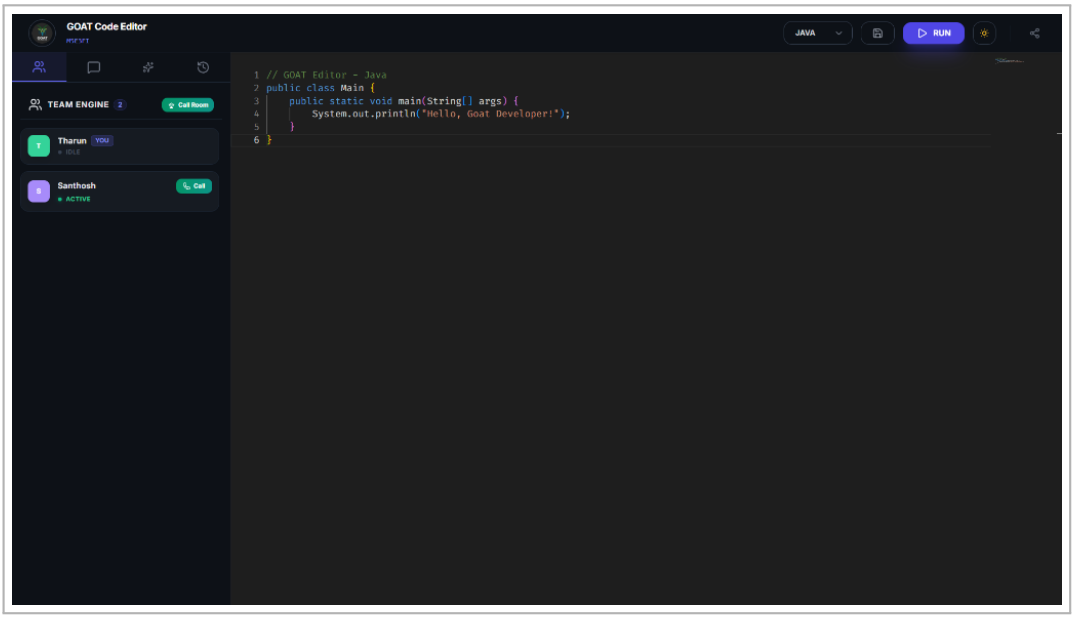


FIGURE B.2: MONACO CODE COMPILER & COLLABORATIVE LIVE TEACHING HUB

(Interface illustration: Standalone microservice deployed at `goatcode-editor.onrender.com` isolating WebRTC audio/video call room and code execution loads)

The GOAT Code Editor and Collaborative Teaching Engine is engineered as an autonomous microservice deployed at `https://goatcode-editor.onrender.com`. Deployed independently from the primary management portal on Vercel, this architectural decoupling isolates CPU-intensive compilation subprocesses, interactive WebRTC peer rooms, and WebSocket signaling during high-concurrency classroom sessions.

Key operational features demonstrated in Figure B.2 include:

- **Monaco Code Editor Core:** Powered by Microsoft's Monaco engine, providing real-time syntax linting, bracket pair colorization, and multi-language compilation for C, C++, Java, and Python.
- **Real-Time Peer Collaboration:** WebRTC media channels enable multi-user collaborative coding, interactive instructor walk-throughs, and peer review rooms with sub-100ms synchronization latency observed under local departmental Wi-Fi networks.
- **Resilient Execution Sandbox:** Integrates local process execution with automated fallback to the Judge0 CE API, providing fault-tolerant compilation during batch laboratory submissions.

FACULTY VERIFICATION MATRIX & CORPORATE PLACEMENT PORTAL

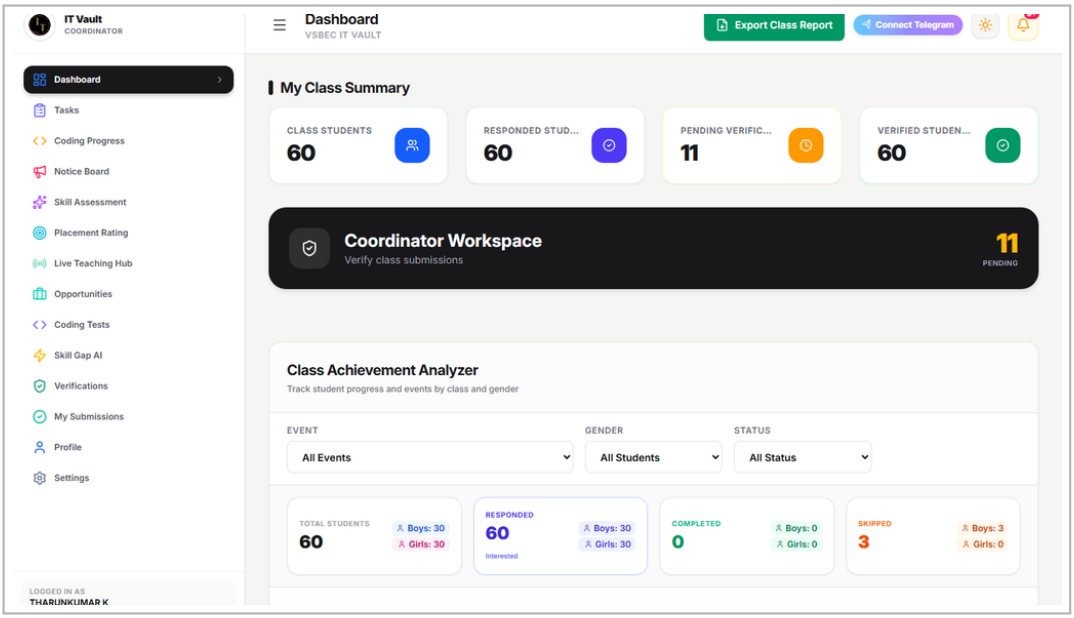


FIGURE B.3: COORDINATOR WORKSPACE & VERIFICATION DASHBOARD

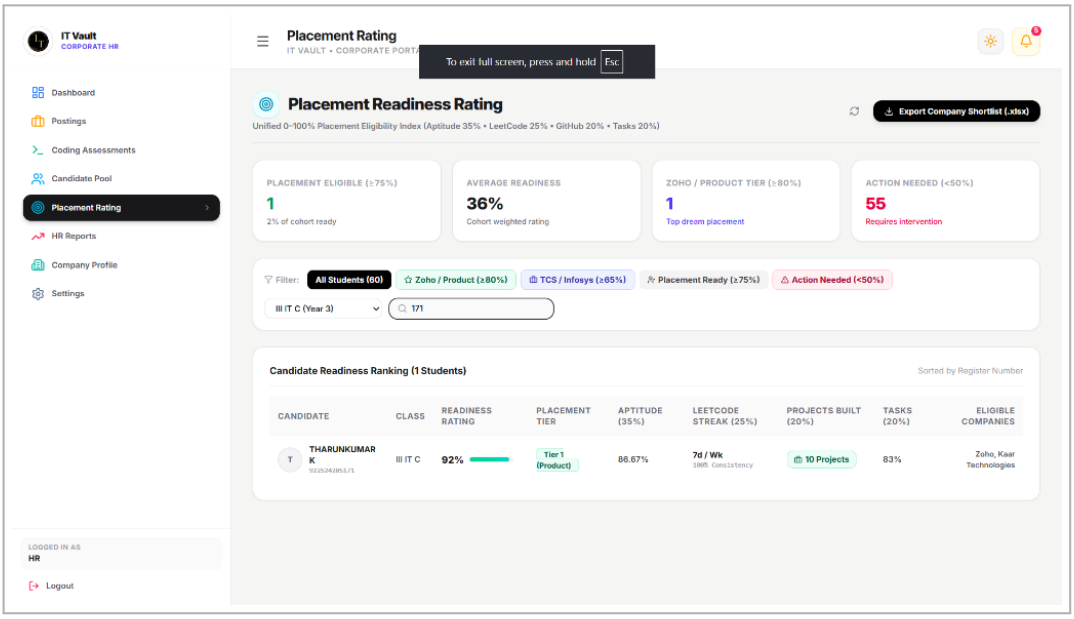


FIGURE B.4: CORPORATE HR PLACEMENT READINESS RATING PORTAL

REFERENCES

1. Jeon, J., Jang, J., and Jung, S. K., (2026). "Development and Classroom Evaluation of a Static-Assessment Programming Practice Platform: Scaffolding Intensity and Subsequent-Task Performance," *IEEE Access*, vol. 14, pp. 66312–66322, doi: 10.1109/ACCESS.2026.3686194.
2. Moreno-Marcos, P. M., Antolín, C. G., Alario-Hoyos, C., Muñoz-Merino, P. J., and Kloos, C. D., (2025). "Integration of Multiple Sources to Anticipate Student Performance Using Learning Analytics," in *Proceedings of the 25th IEEE International Conference on Advanced Learning Technologies (ICALT 2025)*, IEEE, pp. 117–119, doi: 10.1109/ICALT64023.2025.00039.
3. Cho, K., Ali, S. F., Bang, S. J., and Anwar, S., (2024). "Shaking The Silos: Impact of Sequential Live Coding on Students' Performance and Perceptions," in *Proceedings of the 2024 ASEE Annual Conference & Exposition*, ASEE, Portland, OR, doi: 10.18260/1-2–47966.
4. Wasik, S., Antczak, M., Badura, J., Laskowski, A., and Sternal, T., (2018). "A Survey on Online Judge Systems and Their Applications," *ACM Computing Surveys (CSUR)*, vol. 51, no. 1, pp. 1–34, doi: 10.1145/3143560.
5. Sandhu, R. S., Coyne, E. J., Feinstein, H. L., and Youman, C. E., (1996). "Role-Based Access Control Models," *IEEE Computer*, vol. 29, no. 2, pp. 38–47, doi: 10.1109/2.485845.
6. Keuning, H., Jeuring, J., and Heeren, B., (2019). "A Systematic Literature Review of Automated Feedback Generation for Programming Exercises," *ACM Transactions on Computing Education (TOCE)*, vol. 19, no. 1, pp. 1–43, doi: 10.1145/3231711.
7. Zagalsky, A., Feliciano, J., Storey, M. A., Zhao, Y., and Wang, W., (2015). "The Emergence of GitHub as a Collaborative Platform for Education," in *Proceedings of the 18th ACM Conference on Computer Supported Cooperative Work & Social Computing (CSCW)*, ACM, pp. 1906–1917.
8. Atoum, Y., Chen, L., Liu, A. X., Hsu, S. D., and Liu, X., (2017). "Automated Online Exam Proctoring," *IEEE Transactions on Multimedia*, vol. 19, no. 7, pp. 1609–1624, doi: 10.1109/TMM.2017.2656064.
9. Garousi, V., Giray, G., Tüzün, E., Catal, C., and Felderer, M., (2020). "Closing the Gap Between Software Engineering Education and Industrial Needs," *IEEE Software*, vol. 37, no. 2, pp. 68–77.
10. W3C Web Platform Working Group, (2024). "Push API," W3C Candidate Recommendation. [Online]. Available: https://www.w3.org/TR/push-api/.
11. Thomson, M. and Beverloo, P., (2017). "Voluntary Application Server Identification (VAPID) for Web Push," Internet Engineering Task Force (IETF), RFC 8292, doi: 10.17487/RFC8292.

BIBLIOGRAPHY

1. Foundational Software Engineering and Database Textbooks:

- Silberschatz, A., Korth, H. F., and Sudarshan, S., *Database System Concepts*, 7th ed., New York, NY: McGraw-Hill Education, 2019.
- Tanenbaum, A. S. and Van Steen, M., *Distributed Systems: Principles and Paradigms*, 3rd ed., CreateSpace Independent Publishing, 2017.
- Pressman, R. S. and Maxim, B. R., *Software Engineering: A Practitioner's Approach*, 9th ed., New York, NY: McGraw-Hill Education, 2020.
- Gamma, E., Helm, R., Johnson, R., and Vlissides, J., *Design Patterns: Elements of Reusable Object-Oriented Software*, Reading, MA: Addison-Wesley Professional, 1994.
- Martin, R. C., *Clean Architecture: A Craftsman's Guide to Software Structure and Design*, Boston, MA: Prentice Hall, 2017.

2. Cloud Architecture, Runtime Engines, and Web Standards:

- OpenJS Foundation, *Node.js Documentation and Execution Lifecycle*, version 22.x LTS, 2026. [Online]. Available: https://nodejs.org/docs/latest-v22.x/api/.
- PostgreSQL Global Development Group, *PostgreSQL 16 Documentation: Concurrency Control and Row-Level Security Policies*, 2024. [Online]. Available: https://www.postgresql.org/docs/16/.
- Mozilla Developer Network (MDN), *Page Visibility API: Document visibilitychange event and visibilityState property*, 2025. [Online]. Available: https://developer.mozilla.org/en-US/docs/Web/API/Page_Visibility_API.
- Mozilla Developer Network (MDN), *Fullscreen API: Element.requestFullscreen() and Document.fullscreenchange event*, 2025. [Online]. Available: https://developer.mozilla.org/en-US/docs/Web/API/Fullscreen_API.
- Meta Platforms and React Open Source, *React 19 Documentation: Server Components, Actions, and State Primitives*, 2025. [Online]. Available: https://react.dev/.